A Project Report on

An Explainable AI-Based Adaptive Study Planner for Personalized and Stress-Aware Learning

Submitted in partial fulfilment for award of
Bachelor of Technology

Degree

in

CSE (Artificial Intelligence and Machine Learning)

By

S. Sharon (Y22ACM475) **Ch. Mohitha (L23ACM494)**

N.V. Nagalakshmi (Y22ACM459) **G.Sankara Rao (L23ACM495)**

N. Bharath Kumar (Y22ACM458)

Under the guidance of

Dwija Desireddy, M. Tech

Assistant Professor

Department of CSE (Artificial Intelligence and Machine Learning)

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of
CSE (Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that the project report entitled **An Explainable AI-Based Adaptive Study Planner for Personalized and Stress-Aware Learning** being submitted by Somarouthu Sharon (Y22ACM475), Chappidi Mohitha (L23ACM494), Niluri Venkata Nagalakshmi (Y22ACM459), Gottapu Sankara Rao (L23ACM495), and Nerella Bharath Kumar (Y22ACM458) in partial fulfilment for the award of the Degree of Bachelor of Technology in CSE (Artificial Intelligence and Machine Learning) to Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide	**Signature of the HOD**
Dwija Desireddy	**Dr.K.Manideep**
Assistant Professor	**Assoc. Prof. & Head**

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Somarouthu Sharon (Y22ACM475)

Chappidi Mohitha(L23ACM494)

Niluri Venkata Nagalakshmi (Y22ACM459)

Gottapu Sankara Rao (L23ACM495)

Nerella Bharath Kumar (Y22ACM458)

Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Ms**. **Dwija Desireddy,** Assistant Professor, Department of CSE(AIML), for her valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. K. Manideep,** Assoc. Prof. & Head, Department of CSE(AIML) for extending his cooperation and providing the required resources.

We would like to thank our beloved Principal **Dr. B. Srinivasa Rao** for providing the online resources and other facilities to carry out this work.

We would like to sincerely thank our project coordinator **Ms. G. V. Leela Kumari**, Assistant Prof., Dept. of CSE(AIML), for her helpful suggestions in presenting this document.

We extend our sincere thanks to all other teaching faculty and non-teaching staff of the department, who helped directly or indirectly for their cooperation and encouragement.

Somarouthu Sharon (Y22ACM475)
Chappidi Mohitha (L23ACM494)
Niluri Venkata Nagalakshmi (Y22ACM459)
Gottapu Sankara Rao (L23ACM495)
Nerella Bharath Kumar (Y22ACM458)

ABSTRACT

Students today face increasing academic pressure due to ineffective time management, rigid study schedules, and lack of personalized learning support. Traditional planners rely primarily on deadlines and task priority without considering individual productivity patterns, stress levels, or subject-specific weaknesses. Such static approaches often lead to inconsistent performance and academic burnout. This project proposes an AI-Based Adaptive Study Planner that dynamically generates personalized study schedules using behavioural analytics and performance-driven intelligence. The system collects user inputs such as tasks, deadlines, priorities, available study hours, stress levels, and completion history. Using these insights, the system generates optimized study plans that adapt according to performance trends, stress indicators, and historical consistency. The model incorporates self-optimizing workload adjustment, biological rhythm-based scheduling, and automatic prioritization of weak subjects. To ensure sustainable learning, stress-aware logic and Pomodoro-based session structuring are integrated to prevent overload and improve focus. Additionally, the system implements an Explainable AI mechanism that provides schedule reasoning, personalized study suggestions, and motivational feedback, enhancing transparency and user engagement. The application is deployed as a web-based interactive platform with performance tracking and schedule management features. Experimental results indicate improved task consistency, workload balance, and adaptive personalization compared to conventional static study planning tools. The proposed system transforms traditional scheduling into an intelligent, predictive, and user-centered academic assistance framework.

Keywords: Artificial Intelligence, Adaptive Scheduling, Study Planner, Pomodoro Technique, Explainable AI

TABLE OF CONTENTS

ABSTRACT iv
TABLE OF CONTENTS v
LIST OF TABLES vii
LIST OF FIGURES viii
1. Introduction 1
1.1 Background of the Study 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope 3
1.5 Organisation of the Report 3
2. Literature Review 4
2.1 Overview of Existing Work 4
2.2 Comparison of Previous Approaches 5
3. System Analysis 6
3.1 Existing System 6
3.2 Proposed System 6
3.3 Feasibility Study 7
3.4 Requirements 8
3.4.1 Hardware Requirements 8
3.4.2 Software Requirements 8
3.4.3 Functional Requirements 9
3.4.4 Non-Functional Requirements 9
4. Software Design 10
4.1 Architecture Design 10
4.2 UML Diagrams 11
4.2.1 Class Diagram 11
4.2.2 Interaction Diagrams 12
4.2.2.1 Sequence Diagram 12
4.2.2.2 Collaboration Diagram 13
4.2.3 Object Diagram 14
4.2.4 Use Case Diagram 15
4.2.5 Activity Diagram 16
4.2.6 Control Flow Diagram 17
4.3 Database Design 18
4.4 Algorithm / Design Methodology 19
5. Implementation 20
5.1 Technologies Used 20
5.2 Modules Description 21
5.2.1 Task Management Component 21
5.2.2 Predictive Intelligence Component 21
5.2.3 Adaptive Scheduling Component 22
5.2.4 Wellbeing Optimization Component 22

5.2.5 Generative AI Explanation Component 23
5.2.6 Closed-Loop Feedback Learning Mechanism 23
5.2.7 System Support and Interaction Component 23
5.3 Coding 23
5.3.1 Project Folder Structure 23
5.3.2 Backend Implementation 24
5.3.3 Frontend Implementation 30
6. Testing 59
6.1 Test Cases 59
6.2 Testing Methods 59
6.2.1 Unit Testing 60
6.2.2 Integration Testing 60
6.2.3 System Testing 60
6.3 Results and Analysis 60
7. Results and Discussion 61
7.1 Output Screenshots 61
7.2 Performance Analysis 64
7.3 Comparison with Existing System 65
8. Conclusion and Future Work 66
8.1 Summary of the Project 66
8.2 Achievements 66
8.3 Limitations 67
8.4 Future Enhancements 67
9. References 68

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparison of Existing AI-Based Educational Systems	5
Table 4.1	Task Schema	18
Table 4.2	User Progress Schema	19
Table 6.1	Test Cases of AI-Based Adaptive Study Planner	59
Table 7.1	Comparison of Traditional and Proposed Study Planning Systems	65

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig 4.1	Architecture of the Explainable AI-Based Adaptive Study Planner	10
Fig 4.2.1	Class Diagram	12
Fig 4.2.2.1	Sequence Diagram	13
Fig 4.2.2.2	Collaboration Diagram	14
Fig 4.2.3	Object Diagram	15
Fig 4.2.4	Use Case Diagram	16
Fig 4.2.5	Activity Diagram	17
Fig 4.2.6	Control Flow Diagram	18
Fig 5.3.1	Layered Execution Flow of the AI-Based Study Planner	58
Fig 5.3.2	File Interaction Flow of the AI-Based Study Planner	58
Fig 7.1.1	Dashboard Interface of the Study Planner	61
Fig 7.1.2	Task Creation and Task Management Interface	62
Fig 7.1.3	AI-Based Study Plan Generator	62
Fig 7.1.4	Calendar View of Generated Study Schedule	63
Fig 7.1.5	Study Analytics Dashboard	63
Fig 7.1.6	Explainable AI-Based Study Schedule Recommendations	64

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Effective time management is essential for academic success, yet many students struggle to create realistic and sustainable study schedules. Conventional planners and digital calendar applications rely primarily on deadlines and priority levels, offering static scheduling without considering individual productivity patterns, subject-specific weaknesses, stress levels, or peak performance hours. Such rigid approaches often lead to inconsistent performance, inefficient study allocation, and academic burnout.

Advancements in Artificial Intelligence enable the development of adaptive systems capable of learning from user behaviour and personalizing decision-making processes. An intelligent study planner should analyse completion history, productivity trends, stress indicators, and biological rhythm patterns to optimize workload distribution. Incorporating wellness-aware mechanisms is equally critical to ensure that performance improvement does not compromise mental health and long-term sustainability.

The proposed AI-Based Adaptive Study Planner generates personalized, self-optimizing schedules using behavioural analytics and predictive performance metrics such as productivity score, subject weakness score, and schedule confidence score. The system dynamically adjusts workload based on user consistency, detects peak productivity periods through biological rhythm analysis, integrates stress-aware scheduling and structured focus techniques, and provides explainable guidance to enhance transparency and engagement. By combining adaptive intelligence with wellness-centered optimization, the system transforms traditional static planning into a personalized, predictive, and sustainable academic assistance framework.

1.2 PROBLEM STATEMENT

Students today face significant challenges in managing academic workload effectively, not due to lack of effort, but due to the absence of intelligent guidance on how to allocate time, adjust workload, and optimize study efficiency. Existing study planning approaches primarily rely on static scheduling mechanisms based on deadlines and task lists, without incorporating critical human factors such as stress levels, productivity patterns, cognitive fatigue, behavioural consistency, and peak performance periods.

This limitation often results in inefficient study allocation, unrealistic schedules, reduced adherence, increased academic stress, and inconsistent performance outcomes. Traditional planning tools lack adaptive and predictive capabilities to dynamically modify schedules based on measurable productivity trends or subject-specific learning difficulties. Furthermore, they do not provide structured mechanisms to balance academic performance with student wellbeing.

Although research has introduced adaptive learning models and algorithm-based schedulers, comprehensive systems that integrate behavioural analytics, biological rhythm awareness, stress-aware workload optimization, and sustainable learning principles within a unified practical framework remain limited. Therefore, there exists a need for an AI-based adaptive study planning system capable of intelligently adjusting schedules based on performance metrics, behavioral patterns, stress conditions, and overall wellbeing to enhance learning efficiency, improve consistency, and ensure sustainable academic progress.

1.3 OBJECTIVES

The primary objectives of the proposed AI-Based Adaptive Study Planner are:

1. To design and develop an intelligent study scheduling system that generates personalized schedules based on tasks, deadlines, priority levels, and available study hours.
2. To implement behavioural analytics mechanisms that analyse completion history and calculate performance metrics such as productivity score, subject weakness score, and schedule confidence score.
3. To incorporate adaptive scheduling algorithms that dynamically adjust workload based on user performance trends, stress levels, and historical consistency.
4. To integrate biological rhythm detection for identifying peak productivity periods and allocating cognitively demanding subjects during optimal performance hours.
5. To implement wellness-centered optimization through stress-aware scheduling, workload balancing, burnout prevention logic, and structured focus techniques such as the Pomodoro method.
6. To develop an Explainable AI component that provides schedule reasoning, personalized study suggestions, and motivational guidance to enhance transparency and user trust.
7. To design an interactive web-based dashboard that enables task input, performance tracking, schedule visualization, and downloadable study plans.

8. To evaluate system effectiveness by analysing improvements in task consistency, workload balance, and adaptive personalization compared to static scheduling approaches.

1.4 SCOPE

The scope of the proposed AI-Based Adaptive Study Planner is to provide an intelligent, web-based academic scheduling system that generates personalized and performance-driven study plans for individual users. The system focuses on adaptive workload distribution using task parameters, behavioural analytics, stress-level inputs, and biological rhythm-based optimization. It integrates productivity tracking, confidence evaluation, structured focus sessions, and interactive dashboard analytics to support consistent and sustainable academic performance.

The current implementation is designed for single-user interaction within a web application environment and emphasizes adaptive scheduling, stress-aware optimization, and performance visualization. While the system incorporates behavioural learning and self-optimizing mechanisms, features such as multi-user authentication, real-time push notifications, and mobile-native deployment are considered potential future enhancements. The scope of the project is therefore limited to intelligent personalized study planning within a modular web-based framework.

1.5 ORGANISATION OF THE REPORT

This report is organized into multiple chapters for systematic presentation of the proposed AI-Based Adaptive Study Planner. Chapter 1 introduces the project including background, problem statement, objectives, and scope. Chapter 2 presents the literature review and comparison of existing approaches. Chapter 3 discusses system analysis including existing system, proposed system, feasibility study, and requirements. Chapter 4 explains software design including architecture, UML diagrams, database design, and methodology. Chapter 5 describes implementation details and technologies used. Chapter 6 presents testing methodologies and results. Chapter 7 illustrates output screens and performance results. Finally, Chapters 8 and 9 provide conclusion and future scope respectively.

CHAPTER 2

LITERATURE REVIEW

2.1 OVERVIEW OF EXISTING WORK

Artificial Intelligence (AI) has significantly influenced modern educational technologies by enabling personalized learning and intelligent academic support systems. Several studies have explored adaptive learning platforms, real-time activity monitoring, educational data mining, and AI-driven personalized learning approaches to improve student engagement and learning efficiency. AI-based educational software and adaptive learning systems analyze learner interactions and performance patterns to deliver customized learning experiences and instructional recommendations [2]. Deep learning techniques have also been employed in adaptive e-learning environments to monitor student activities and engagement in real time, enabling improved understanding of learner behavior and participation [3]. Technology-enhanced learning environments such as robotics-based educational tools further support the development of cognitive skills and self-regulated learning among students [4]. Additionally, data mining and big data analytics have been applied to analyze large-scale educational datasets and identify patterns that support predictive decision-making in academic environments [5]. Various productivity and engagement applications have also been introduced to help students monitor their study habits and maintain a balance between academic tasks and daily routines [6]. Furthermore, deep neural network models have been utilized to detect and analyze behavioral patterns in real-time monitoring systems [7], while AI-based personalized learning platforms dynamically adapt learning materials and instructional paths based on student progress and performance [8].

Despite these advancements, many existing systems primarily focus on personalized content delivery, learner engagement monitoring, or educational data analysis rather than providing intelligent study scheduling and structured academic task management. Most available tools rely on static scheduling mechanisms or domain-specific adaptive learning approaches that do not dynamically adjust study plans according to student productivity patterns, workload distribution, or learning behavior. Recent research has proposed AI-powered smart study planners that integrate machine learning algorithms, behavioral analytics, and adaptive scheduling techniques to generate personalized study schedules tailored to learner preferences, goals, and performance data. Such systems demonstrate improvements in learning efficiency and academic engagement when compared to traditional manual planning methods.

However, there remains a need for more comprehensive systems that effectively combine behavioral analysis, adaptive scheduling, and intelligent workload management within a unified learning support framework [9].

2.2 COMPARISON OF PREVIOUS APPROACHES

Table 2.1: Comparison of Existing AI-Based Educational Systems

Study / Tool	Methodology	Limitation	Ref
Knewton	AI-driven adaptive learning platform	Does not provide integrated study scheduling or daily planning	[2]
Smart Sparrow	Real-time adaptive e-learning with learner feedback	Lacks predictive scheduling and automated time management support	[3]
DreamBox Learning	Adaptive mathematics learning through intelligent pacing	Limited to mathematics; lacks study planning features	[4]
Big Data Educational Analytics	Data mining techniques for analyzing educational datasets	Focuses on large-scale data analysis rather than personal study scheduling	[5]
MyStudyLife	Calendar-based study planner with reminders and task tracking	Not AI-driven and cannot adapt to learning behavior	[6]
Deep Neural Monitoring System	Behavioral monitoring using deep neural networks	Primarily designed for safety monitoring, not study management	[7]
Squirrel AI	AI-based personalized learning platform	Focuses on content delivery rather than intelligent scheduling	[8]

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Current study planning methods primarily include manual timetable creation, digital calendar applications, and basic task management tools. These systems allow users to create task lists, assign deadlines, and schedule reminders. While they provide basic organizational support, they operate using static scheduling logic and do not adapt to user behavior or performance patterns.

Most existing systems allocate study sessions solely based on deadlines and manually assigned priorities. They do not analyze historical task completion data, productivity consistency, stress levels, or subject-specific performance trends. As a result, generated schedules remain rigid and fail to adjust dynamically according to changes in workload or student wellbeing.

Furthermore, conventional planners lack predictive or adaptive mechanisms to optimize workload distribution. They do not provide structured workload balancing, burnout prevention strategies, or behavioral performance tracking necessary for sustainable academic progress.

Limitations of Existing System

The major limitations of current study planning systems are:

- Static deadline-based scheduling without adaptive intelligence
- No behavioral learning from task completion history
- Absence of productivity or performance scoring mechanisms
- Lack of stress-aware workload adjustment
- No biological rhythm-based task allocation
- No structured burnout prevention strategy
- Limited motivational feedback mechanisms
- Inability to dynamically redistribute workload based on real-time performance

3.2 PROPOSED SYSTEM

The proposed system is an AI-Based Adaptive Study Planner designed to generate personalized and dynamically optimized study schedules. Unlike traditional planners, the system integrates behavioral analytics, performance-based scoring, stress-aware workload adjustment, and biological rhythm detection to enhance both academic efficiency and student wellbeing.

The system collects task details including subject, deadline, priority level, available study hours, stress level, and task completion history. Based on this data, it computes performance indicators such as Productivity Score and Schedule Confidence Score to evaluate user consistency and learning trends.

Using these metrics, the adaptive scheduling mechanism dynamically allocates study sessions, prioritizes weaker subjects, and adjusts workload intensity according to performance and stress conditions. The system incorporates wellness optimization strategies by regulating session duration and integrating structured focus intervals through the Pomodoro technique.

Additionally, an interactive dashboard provides real-time analytics including subject-wise progress tracking, completion statistics, performance visualization, and schedule management features. This integrated approach transforms static study planning into an adaptive, performance-driven, and wellness-centered academic assistance system.

Advantages of Proposed System

- Adaptive scheduling based on behavioral performance metrics
- Productivity and confidence score evaluation
- Stress-aware workload optimization
- Biological rhythm-based task allocation
- Dynamic workload redistribution
- Structured Pomodoro integration for enhanced focus
- Real-time performance visualization through analytics dashboard
- Wellness-centered burnout prevention strategy
- Modular and scalable web-based implementation

3.3 FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of implementing the proposed AI-Based Adaptive Study Planner system.

Technical Feasibility

The proposed system is technically feasible as it is developed using widely adopted modern web technologies such as React.js, TypeScript, Node.js, and Express.js. These technologies provide sufficient flexibility and support for implementing adaptive scheduling algorithms, AI integration, and interactive dashboards. The required development tools and frameworks are readily available and compatible with current computing systems.

Economic Feasibility

The project is economically feasible because it does not require expensive hardware or proprietary software licenses. The development tools used are open-source and freely available. Deployment and maintenance costs are minimal since the system is web-based and can run on standard computing devices.

Operational Feasibility

The proposed system is operationally feasible because it is designed with a simple and user-friendly interface that can be easily operated by students without requiring technical expertise. The workflow of the application aligns with the natural academic planning habits of users, making it practical for real-world adoption.

3.4 REQUIREMENTS

3.4.1 Hardware Requirements

The proposed AI-Based Adaptive Study Planner is a web-based application and does not require specialized hardware. The minimum hardware requirements for development and deployment are as follows:

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB (8 GB recommended for development)
- Storage: Minimum 10 GB of available disk space
- Internet Connection: Stable broadband connection
- Display: Standard monitor with minimum resolution of 1366 × 768

For development purposes, a system with higher processing power and memory is recommended to ensure smooth execution of frontend compilation and application rendering.

3.4.2 Software Requirements

The system is developed using modern web technologies with a TypeScript-based frontend architecture. The required software components include:

- Operating System: Windows / Linux / macOS
- Frontend Framework: React.js
- Programming Language: TypeScript
- Runtime Environment: Node.js
- Package Manager: npm
- Build Tool: Vite / Create React App
- Code Editor: Visual Studio Code
- Web Browser: Google Chrome / Mozilla Firefox
- Version Control System: Git

3.4.3 Functional Requirements

The system shall:

- Allow users to create, edit, and delete study tasks
- Accept task details including subject name, deadline, priority level, and estimated duration
- Allow users to input daily available study hours
- Record user-reported stress levels
- Track task completion status over time
- Generate adaptive study schedules dynamically
- Calculate Productivity Score based on completion rate and consistency
- Calculate Schedule Confidence Score based on historical adherence patterns
- Adjust workload dynamically based on stress level inputs
- Allocate tasks based on biological rhythm-based productivity trends
- Integrate structured Pomodoro sessions for focused study intervals
- Display subject-wise and weekly performance analytics
- Provide calendar-based schedule visualization
- Allow monitoring of progress and performance through dashboard interface

3.4.4 Non-Functional Requirements

- Performance: The system shall generate study schedules within acceptable response time and render UI efficiently.
- Usability: The interface shall be intuitive, responsive, and user-friendly.
- Reliability: The system shall maintain stable performance without data inconsistency during usage.
- Scalability: The modular architecture shall support future integration of additional features.
- Maintainability: The codebase shall follow structured and modular design principles.
- Security: Basic input validation and data handling mechanisms shall be implemented.
- Compatibility: The application shall function properly across modern web browsers.

CHAPTER 4

SOFTWARE DESIGN

4.1 ARCHITECTURE DESIGN

The architecture of the proposed AI-Based Adaptive Study Planner is designed using a modular layered structure to ensure scalability, maintainability, and efficient interaction among system components. The system is divided into multiple interconnected modules including Task Management, Predictive Intelligence, Adaptive Scheduling Engine, Wellbeing Optimization, Generative AI Explanation Module, Feedback Learning Mechanism, and User Dashboard Interface.The Task Management Module collects and stores academic tasks entered by the user. The Predictive Intelligence Module evaluates productivity, weakness, and confidence scores based on user behavior and historical performance. The Adaptive Scheduling Engine processes these metrics to generate optimized schedules while balancing workload and deadlines.The Wellbeing Optimization Module integrates stress-aware scheduling and Pomodoro-based session planning to prevent burnout. The Generative AI Module enhances the system by providing schedule explanations, recommendations, and motivational support. Finally, the Dashboard Interface presents schedules, analytics, and insights to users in an interactive manner.

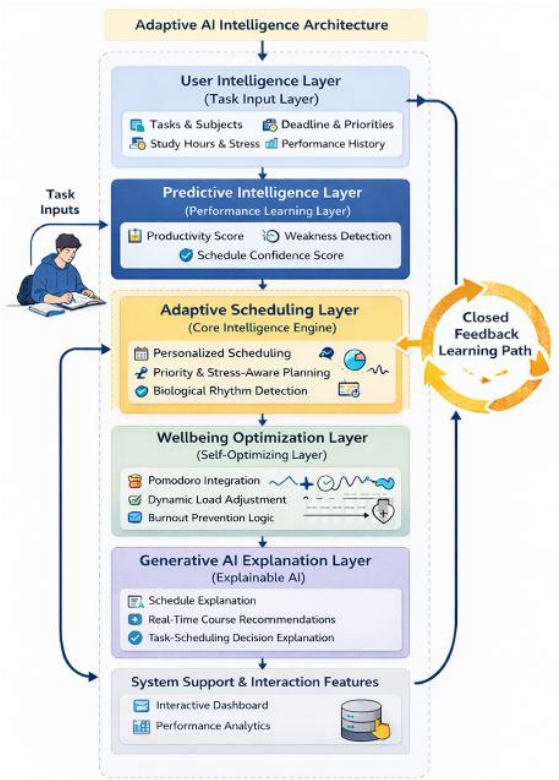

Fig 4.1: Architecture of the Explainable AI-Based Adaptive Study Planner

4.2 UML DIAGRAMS

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. We use UML diagrams to show the behavior and structure of a system.UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

4.2.1 Class Diagram

Objective:

Define the static structure of the AI-Based Adaptive Study Planner system, including classes, their attributes, methods, and the relationships between system components.

Description:

- **Classes:** Task, AIScheduleItem, UserProgress, SchedulerEngine, PomodoroSession, GenAIService, AnalyticsModule

Task – Represents an academic task created by the user and stored in the system.

- **Attributes:** id, title, description, subject,priority,dueDate,timeEstimate,completed, createdAt
- **Methods:** createTask(), updateTask(), deleteTask(), markCompleted()

AIScheduleItem – Represents a scheduled study session generated by the scheduling system.

- **Attributes:** task, suggestedTime, sessionDuration, explanation, recommendedCourse
- **Methods:** generateSchedule(), updateSchedule()

UserProgress – Stores the user's behavioral performance metrics.

- **Attributes:** currentStreak, bestStreak, totalCompleted, lastCompletedDate, stressLevel
- **Methods:** updateProgress(), calculateProductivityScore(), calculateScheduleConfidence()

SchedulerEngine – Core module responsible for generating adaptive study schedules.

- **Attributes:** tasks, dailyHours, stressLevel, productivityScore
- **Methods:** suggestStudySchedule(), adjustScheduleForStress(), detectBestStudyHour()

PomodoroSession – Handles structured focus sessions using the Pomodoro technique.

- **Attributes:** id, taskId, duration, startTime, completed
- **Methods:** startSession(), completeSession(), saveSession()

GenAIService – Provides AI-based explanations, motivational messages, and study resource recommendations.

- **Attributes:** apiKey, modelName
- **Methods:** generateMotivation(), generateExplanation(), generateResourceSuggestion(), enhanceSchedule()

AnalyticsModule – Processes study data and generates performance analytics.

- **Attributes:** tasks, completionRate, studyTime
- **Methods:** generateAnalytics(), calculateStatistics(), visualizeProgress()

Relationships:

- SchedulerEngine uses Task data to generate schedules.
- AIScheduleItem is generated by SchedulerEngine.
- UserProgress provides productivity and stress metrics to SchedulerEngine.
- PomodoroSession records study sessions and updates progress tracking.
- GenAIService enhances generated schedules with explanations and resources.
- AnalyticsModule processes task and session data to generate insights.

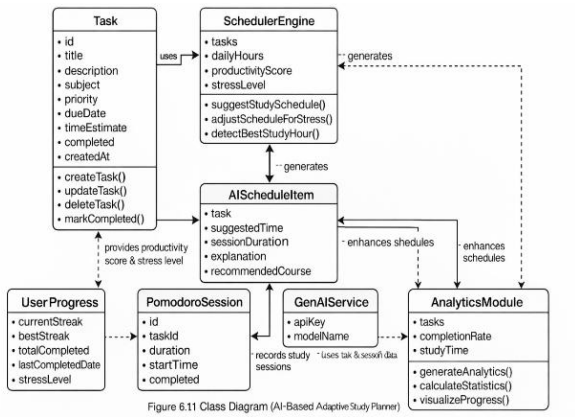

Fig 4.2.1. Class Diagram

4.2.2 Interaction Diagrams

4.2.2.1 Sequence Diagram

Objective:

Show how objects interact in a particular sequence to generate an adaptive study schedule.

- **Actors:** User, Dashboard Interface, Task Manager, Scheduler Engine, GenAI Service, Analytics Module
- **Sequence:**

1. **User** enters study tasks, deadlines, priority levels, available study hours, and stress level.

2. **Dashboard Interface** sends the task information to the Task Manager.
3. **Task Manager** stores and organizes the user's tasks.
4. **Scheduler Engine** retrieves tasks and calculates productivity score, subject weakness score, and priority weights.
5. **Scheduler Engine** generates an optimized study schedule based on deadlines, priority levels, stress conditions, and available study hours.
6. **GenAI Service** enhances the schedule by generating explanations and recommending learning resources.
7. **Analytics Module** processes task and performance data to generate study insights.
8. **Dashboard Interface** displays the AI-generated schedule, explanations, and analytics for the user.

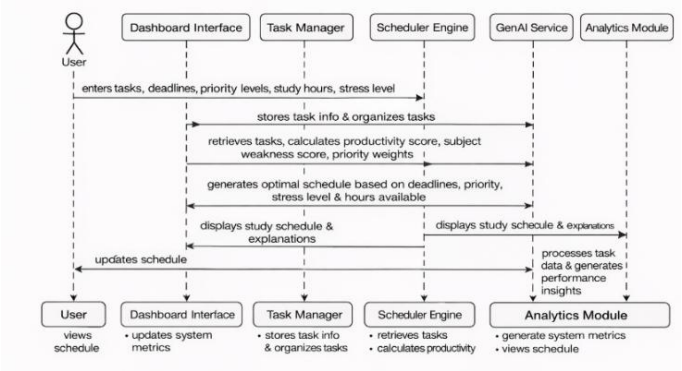

Fig.4.2.2.1 Sequence Diagram

4.2.2.2 Collaboration Diagram

Objective:

Show how system components collaborate with each other to complete the task of generating an adaptive study schedule.

Description:

- **Actors:** User, Dashboard Interface, Task Manager, Scheduler Engine, GenAI Service, Analytics Module
- **Interactions:**

1. **User** enters study tasks, deadlines, available study hours, stress level, and priority levels.
2. **Dashboard Interface** forwards task information to the Task Manager.
3. **Task Manager** organizes and stores the user's study tasks.
4. **Scheduler Engine** retrieves task data and calculates productivity score, subject weakness score, and priority weights.

5. **Scheduler Engine** generates an adaptive study schedule considering deadlines, priority levels, and stress conditions.
6. **GenAI Service** enhances the schedule by adding explanations and recommended study resources.
7. **Analytics Module** processes study data to generate productivity insights and performance analytics.
8. **Dashboard Interface** displays the generated schedule and analytics to the user.

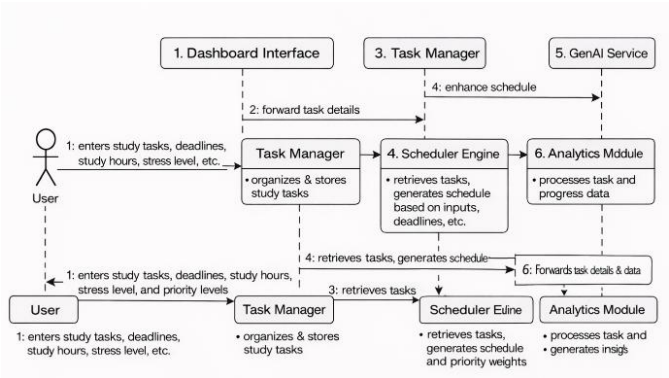

Fig. 4.2.2.2 Collaboration Diagram

4.2.3 Object Diagram

Objective:

Represent a snapshot of objects in the AI-Based Adaptive Study Planner system at a particular moment and illustrate how instances of classes interact with each other.

- **Objects:** User, Task, StudySchedule, SchedulerEngine, GenAIService, AnalyticsModule
- **Interactions:**

1. **User object** creates task instances by entering study information such as subject, deadline, priority level, and estimated duration.
2. **Task objects** store task details including subject name, deadline, and completion status.
3. **SchedulerEngine object** retrieves task instances and evaluates scheduling parameters such as productivity score, priority level, and subject weakness.
4. **StudySchedule object** represents the generated schedule containing study sessions and suggested time slots.

5. **GenAIService object** enhances the generated schedule with explanations and recommended learning resources.
6. **AnalyticsModule object** analyzes task completion data to generate insights about user productivity and performance.
7. **User object** views the generated schedule and analytics through the dashboard interface.

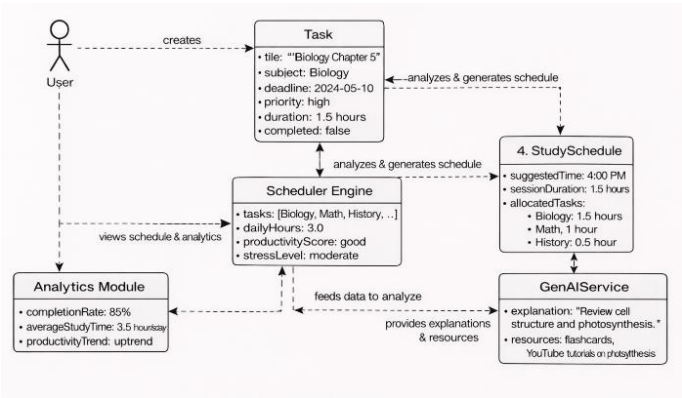

Fig. 4.2.3 Object Diagram

4.2.4 Use Case Diagram

Illustrate the functional interactions between the user and the AI-Based Adaptive Study Planner system by representing the key system functionalities.

- **Actor:** User
- **Use Cases:**

1. **Create Study Tasks** – The user enters tasks by providing subject name, deadline, priority level, and estimated study duration.
2. **Manage Tasks** – The user can edit, update, delete, or mark tasks as completed.
3. **Generate Adaptive Study Schedule** – The system generates an optimized schedule using task details, productivity score, and stress level.
4. **View AI Explanations and Resource Suggestions** – The system explains scheduling decisions and suggests learning resources.
5. **Track Study Progress** – The system tracks task completion, study streaks, and productivity metrics.
6. **View Analytics Dashboard** – The user can view graphs and statistics showing study progress and performance.
7. **Update Stress Level** – The user inputs stress level to allow the system to adjust workload intensity.

8. **View Generated Study Schedule** – The system displays scheduled study sessions with recommended resources.

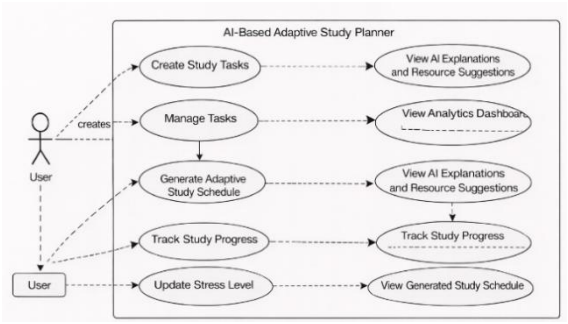

Fig. 4.2.4 Use Case Diagram

4.2.5 Activity Diagram

Represent the workflow of the AI-Based Adaptive Study Planner system by illustrating the sequence of activities involved in generating an adaptive study schedule from user input to schedule visualization.

- **Activities:** User Input, Task Processing, Schedule Generation, AI Enhancement, Analytics Processing, Schedule Display
- **Flow:**

1. **User** starts the application and enters study tasks including subject name, deadline, priority level, available study hours, and stress level.
2. The **Task Manager** receives and stores the entered task information for further processing.
3. The **Scheduler Engine** retrieves the stored tasks and analyzes scheduling parameters such as productivity score, subject weakness score, deadline urgency, and priority level.
4. The **Scheduler Engine** generates an adaptive study schedule by allocating tasks across available study hours while considering stress conditions and workload balance.
5. The **GenAI Service** enhances the generated schedule by providing explanations for scheduling decisions and recommending relevant learning resources.
6. The **Analytics Module** processes task completion data and generates performance insights such as completion rate, productivity trends, and subject-wise progress.
7. The **Dashboard Interface** displays the generated study schedule, explanations, and analytics for the user.

8. **User** follows the schedule and updates task completion status, which feeds back into the system to improve future scheduling decisions.

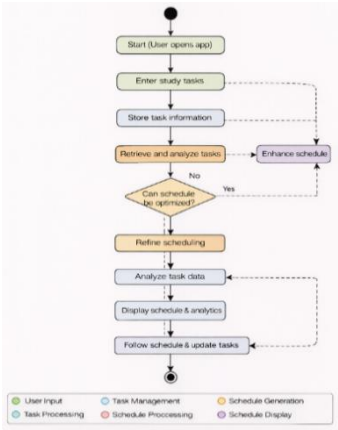

Fig. 4.2.5 Activity Diagram

4.2.6 Control Flow Diagram

Represent the flow of control within the AI-Based Adaptive Study Planner system from user input to schedule generation and result visualization.

Description:

- **Components:** User, Task Manager, Scheduler Engine, GenAI Service, Analytics Module, Dashboard Interface
- **Flow:**

1. **User** enters study tasks, deadlines, priority levels, available study hours, and stress level through the dashboard interface.
2. **Task Manager** stores the entered tasks and organizes them for scheduling.
3. **Scheduler Engine** retrieves stored tasks and evaluates scheduling parameters including deadline urgency, priority levels, productivity score, and subject weakness score.
4. **Scheduler Engine** generates an adaptive study schedule considering available study hours and stress conditions.
5. **GenAI Service** enhances the generated schedule by adding explanations and recommended learning resources.
6. **Analytics Module** analyzes task completion data and generates productivity insights.
7. **Dashboard Interface** displays the generated study schedule, explanations, and analytics to the user.
8. **User** follows the study schedule and updates task completion status, which improves future scheduling decisions.

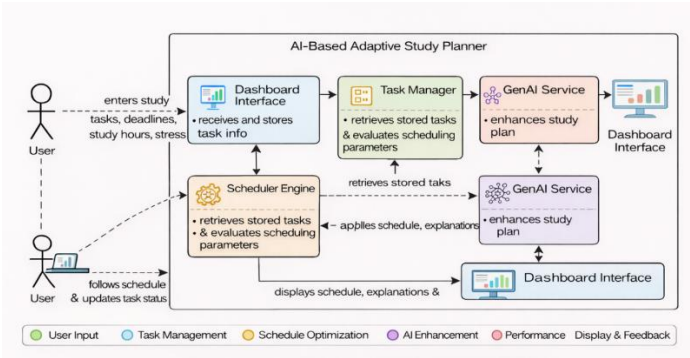

Fig. 4.2.6 Control Flow Diagram

4.3 DATABASE DESIGN

The proposed AI-Based Adaptive Study Planner does not use a traditional relational database. Instead, the system utilizes browser-based local storage mechanisms to manage and persist application data. This lightweight data storage approach ensures simplicity and efficiency for single-user web application deployment.

The primary data entities stored within the system include Tasks, User Progress, Pomodoro Sessions, and Generated Study Schedules.

Table 4.1 Task Schema

Field Name	Data Type	Description
id	String	Unique identifier for task
title	String	Task title
subject	String	Subject name
dueDate	Date	Deadline of task
priority	String	Priority level
timeEstimate	Number	Estimated duration
completed	Boolean	Completion status

Table 4.2 User Progress Schema

Field Name	Data Type	Description
currentStreak	Number	Current study streak
bestStreak	Number	Highest streak
totalCompleted	Number	Completed tasks
stressLevel	Number	User stress indicator

4.4 ALGORITHM / DESIGN METHODOLOGY

The proposed system follows an adaptive scheduling algorithm that dynamically generates study schedules based on task urgency, user productivity, and stress levels.

Scheduling Methodology

1. User enters tasks, deadlines, priorities, and available study hours.
2. The Task Management Module stores the task information.
3. Productivity Score and Schedule Confidence Score are calculated from task completion patterns.
4. Subject weakness and priority weights are determined.
5. The Adaptive Scheduling Engine prioritizes tasks based on urgency and behavioral metrics.
6. Stress-aware adjustment modifies workload intensity according to user stress level.
7. Biological rhythm analysis detects the user's peak productivity periods.
8. The final optimized schedule is generated and displayed to the user.

This algorithm ensures intelligent workload distribution while maintaining student wellbeing and maximizing academic productivity.

CHAPTER 5
IMPLEMENTATION

5.1 TECHNOLOGIES USED

The proposed AI-Based Adaptive Study Planner is developed using modern web technologies and AI-integrated development frameworks to ensure scalability, responsiveness, and intelligent functionality. The technologies used in the implementation of the system are described below:

React.js

React.js is used as the frontend JavaScript library for building the user interface of the application. It enables component-based development, efficient rendering, and dynamic user interaction within the web application.

TypeScript

TypeScript is used as the primary programming language for frontend development. It enhances JavaScript by providing static typing, better debugging capabilities, and improved code maintainability.

Node.js

Node.js serves as the runtime environment for executing backend JavaScript code. It enables server-side processing and API communication between the frontend and backend services.

Express.js

Express.js is used as the backend framework to manage server creation, API routing, and communication between frontend modules and AI service components.

OpenRouter API / Large Language Model Integration

The OpenRouter API is integrated into the system to provide generative AI functionalities such as schedule explanation, motivational feedback, study resource recommendations, and intelligent schedule enhancement.

HTML / CSS

HTML and CSS are used for structuring and styling the application interface to provide a visually appealing and responsive user experience.

Local Storage

Browser local storage is used for lightweight client-side data persistence, enabling storage of tasks, progress metrics, schedules, and Pomodoro session history.

Visual Studio Code

Visual Studio Code is used as the integrated development environment (IDE) for coding, debugging, and project management during development.

5.2 MODULES DESCRIPTION

5.2.1 Task Management Component (User Intelligence Layer)

Among the different elements of the system, Task Management Modules acts as the system's data base. The user will enter the academic work, and this function will collect and maintain the data. Moreover, it keeps the data in a standard format for use by other modules of the system. Tasks are stored using the data structure:(Task ID, Subject Name, Deadline, Priority, Estimated Study Duration, Status).Functions include Making and changing tasks and other characteristics of the task, Erasing and confirming the completion of tasks. With an active task management system, you'll be able to analyze procrastination, consistency, and time of work. Once the tasks are captured and stored, this module offers task data.

5.2.2 Predictive Intelligence Component

The predictive intelligence uses the activity of the user and the historical completion of tasks to provide a view of the user. The Productivity Score of system is estimated based on task fulfillment, completion rate, overdue tasks, consistency and the ability to complete items in the planned study time. A user's **productivity score** is measured based on the overall engagement and motivation levels alongside the level of diligence. Compared to scheduling light tasks for a less productive user, the more productive user consistently performs heavier tasks.In the same manner, a **Schedule Confidence Score** indicates how likely is the user to be able to follow along with the schedule. The confidence score is estimated by analysing the finish behaviour, overdue tasks and stress level.

In order to assess which specific subject or task the user is falling short in, **weakness detection** is carried out. Functions that analyse the various aspects with a view to revealing a comparatively low completion rate for certain tasks or daily time spent.

5.2.3 Adaptive Study Scheduling Component (Core Intelligence Engine)

Proposed system's core intelligence is provided by the Adaptive Study Scheduling Component. The system is aimed at generating individualised study schedules by varying the workload automatically depending on academic priorities, cognitive capacity and user performance.The process of scheduling begins with retrieving incomplete tasks from the Task Managment Module. Subsequently, daily study time is checked for schedule feasibility.

Assessment of Task Importance: Priority weight W_i is assigned to each task based on urgency of deadline and priority. Scheduling importance is greater for tasks of higher urgency.

Adjustment Based on Stress: The intensity of the workload adjusts depending on the user's stress levels. As stress increases the intensity of the session reduced.

Productivity-Based Load Adjustment: Patterns of user productivity are examined to find feasible workloads. If productivity is good, then more tasks may get scheduled and if it is poor, the schedule will be lighter.

Biological Rhythm Analysis: The system assesses efficiency patterns according to the time of the day (E_t) to identify periods of concentration peaks. We schedule more demanding tasks during times when the user is historically more focused

To determine optimal task allocation, the system computes a unified scheduling score:

$$S_i = W_i \times (\lambda_1 PS + \lambda_2 ECC + \lambda_3 E_t)$$

Where , **PS** represents productivity score , **ECC** represents effective cognitive capacity derived from stress levels , $\mathbf{E_t}$ represents time-of-day efficiency and coefficients $\lambda_1, \lambda_2, \lambda_3$ control the influence of these factors.

Tasks with greater scheduling score are prioritized first during schedule generation provided total assigned study time is not more than available daily study time.

5.2.4 Wellbeing Optimization Component

The Wellbeing Optimization Component aims to ensure that the study schedule remains balanced and sustainable in order to prevent overload and fatigue. To achieve this, the Pomodoro study technique, workload adjustment mechanisms, and burnout prevention strategies are integrated into the system.

Study sessions are modeled using the **Pomodoro technique**, which consists of short intervals of focused study followed by brief breaks to avoid mental fatigue while maintaining concentration.The component also performs workload adjustment based on stress awareness by considering the user-reported stress level and cognitive capacity. As stress levels increase, the system either reduces study intensity or shortens study sessions so that the schedule remains realistic and achievable. This functionality helps users maintain a balanced learning routine while protecting their mental well-being.

5.2.5 Generative AI Explanation Component

The Generative AI Explanation Component enhances the transparency and usability of the study planner by providing interpretable insights about generated schedules.

The component performs two key functions:

Explainable Scheduling – Explaining how and why schedules and tasks are generated

Feedback on Personalized Study Guidance – Based on performance on the task, recommending learning material

Through its use of legit reasoning, the system does not just generate schedules but helps students to improve their performance as well.

5.2.6 Closed-Loop Feedback Learning Mechanism

The proposed system incorporates a closed-loop feedback learning mechanism that continuously improves scheduling performance using user interaction data.After each study session, task completion status, productivity patterns, and performance data are recorded. The metrics will improve productivity scores, cognitive capacity estimates, and time-efficiency patterns.The system generates personalised study schedules by continuing to adapt to the way a user studies, with the feedback process.

5.2.7 System Support and Interaction Component

This acts as an interface that users use to interact with the planner.It has interactive dashboard that allows users to view schedules , track task progress , and analyse performance insights .It helps users observe their study behavior and assess whether the generated schedules are actually helping them study better.

5.3 CODING

5.3.1 Project Folder Structure

The proposed system is organized into two main parts: **Backend** and **Frontend**.

The backend is responsible for handling AI-related services and API communication, while the frontend manages the user interface, navigation, scheduling features, analytics, and overall interaction with the system. This modular structure helps in better development, maintenance, and future enhancement of the application.

A. Backend

The backend module contains the server-side logic of the system. It is mainly responsible for processing AI-related requests, generating responses, and connecting the frontend with the language model services.

Files included in Backend:

- `server.js`
- `routes/genai.js`
- `services/openaiService.js`

B. Frontend

The frontend module contains the client-side part of the application. It is responsible for displaying the study planner interface, managing navigation between pages, storing and processing study data, and presenting AI-generated schedules and analytics to the user.

Main files included in Frontend:

- `main.tsx`
- `app/App.tsx`
- `app/routes.ts`
- `app/lib/study-data.ts`
- `app/pages/AISchedule.tsx`

Frontend Components:

- `app/components/Dashboard.tsx`
- `app/components/Calendar.tsx`
- `app/components/Analytics.tsx`
- `app/components/StudyPlan.tsx`
- `app/components/Tasks.tsx`
- `app/components/Settings.tsx`
- `app/components/Root.tsx`
- `app/components/NotFound.tsx`

Reusable UI Components:

- `app/components/ui/card.tsx`
- `app/components/ui/badge.tsx`
- `app/components/ui/button.tsx`
- `app/components/ui/progress.tsx`

C. Styling

The styling section contains the design-related file used to maintain the visual appearance of the application.

Style file:

- `styles/index.css`

5.3.2 Backend Implementation

File: `routes/genai.js`

Purpose:

Handles all backend API routes for AI features in the study planner. It accepts requests from the frontend, sends data to the OpenAI service layer, and returns AI-generated responses.

Main Functions:

- Generates motivational messages
- Explains schedule decisions
- Suggests learning resources
- Enhances schedules using AI
- Provides schedule insights

Code Summary:

The file imports express and AI service functions from openaiService.js, then creates an Express router. All routes use POST since data is sent in the request body.

Routes Included:

- `/motivation` – returns motivational messages
- `/explanation` – explains why the schedule was created that way
- `/resources` – suggests study resources
- `/enhance-schedule` – improves the schedule with AI explanations and recommended courses, while also validating input and using fallback values if needed
- `/schedule-insight` – provides overall schedule insights

Error Handling:

Each route uses try-catch to prevent crashes and return proper error messages if something goes wrong.

Code:

```
import express from "express";
import {
  generateMotivation,
  generateExplanation,
  generateResourceSuggestion,
  enhanceSchedule,
  generateScheduleInsight
} from "../services/openaiService.js";
const router = express.Router();
/* -------- Motivation -------- */
router.post("/motivation", async (req, res) => {
  try {
    const message = await generateMotivation(req.body);
    res.json({ message });
  } catch (error) {
    console.error("Motivation generation error:", error);
    res.status(500).json({
      error: "Failed to generate motivation message"
    });}});
/* -------- Schedule Explanation -------- */
router.post("/explanation", async (req, res) => {
  try {
```

```
    const message = await generateExplanation(req.body);
    res.json({ message });
  } catch (error) {
    console.error("Explanation generation error:", error);
    res.status(500).json({
      error: "Failed to generate explanation"
    });}});
/* -------- Resource Suggestion -------- */
router.post("/resources", async (req, res) => {
  try {
    const message = await generateResourceSuggestion(req.body);
    res.json({ message });
  } catch (error) {
    console.error("Resource suggestion error:", error);
    res.status(500).json({
      error: "Failed to generate resource suggestions"
    });}});
/* -------- Enhanced Schedule (MAIN AI FEATURE) -------- */
router.post("/enhance-schedule", async (req, res) => {
  try {
    const { schedule } = req.body;
    if (!schedule || !Array.isArray(schedule)) {
      return res.status(400).json({
        error: "Invalid schedule format"
      }); }
    const aiData = await enhanceSchedule(req.body);
    console.log("AI DATA RETURNED:");
    console.log(aiData);
    const merged = schedule.map((item, index) => {
  const ai = aiData[index];
  return {
    ...item,
    explanation:
      ai?.explanation ||
      "Scheduled based on priority, subject weakness, productivity level
and stress level.",
    recommendedCourse:
      ai?.recommendedCourse ||
      `YouTube course on ${item.task.subject}`
  };});
    res.json(merged);
  } catch (error) {
    console.error("Schedule enhancement error:", error);
    res.status(500).json({
      error: "AI enhancement failed"
    });}});
/* -------- Schedule Insight -------- */
router.post("/schedule-insight", async (req, res) => {
  try {
    const message = await generateScheduleInsight(req.body);
    res.json({ insight: message });
  } catch (error) {
    console.error("Insight generation error:", error);
```

```
    res.status(500).json({
      error: "Insight generation failed"
    });}});
export default router;
```

File: `services/openaiService.js`

Purpose:

Handles all AI-related backend logic of the study planner. It communicates with the OpenRouter API, sends prompts to the language model, and returns AI-generated responses for different features.

Overview:

This file acts as the AI service layer, keeping AI logic separate from route files for better organization, readability, and maintainability. It uses node-fetch to make HTTP requests.

Main Components:

- **`getAPIKey()`** – checks and returns the API key from environment variables
- **`callLLM(prompt)`** – reusable function that sends prompts to OpenRouter API and returns generated text
- **`generateMotivation(data)`** – creates short motivational messages
- **`generateExplanation(data)`** – explains why a task was scheduled
- **`generateResourceSuggestion(data)`** – suggests study resources for a subject
- **`enhanceSchedule(data)`** – enriches each task with explanations and recommended courses using AI
- **`generateScheduleInsight(data)`** – gives an overall insight about the generated study schedule

Special Feature:

The `enhanceSchedule()` function is the core AI feature. It sends schedule details to the model, expects a JSON array response, removes markdown formatting if needed, extracts valid JSON, and safely parses it.

Error Handling:

The file includes API key validation, response checking, and safe JSON parsing. If any issue occurs, fallback handling prevents the application from crashing.

Code:

```
import fetch from "node-fetch";
/* ---------------- HELPER ---------------- */
function getAPIKey() {
  if (!process.env.OPENROUTER_API_KEY) {
    throw new Error("OPENROUTER_API_KEY missing in .env file");
```

```
  }
  return process.env.OPENROUTER_API_KEY;
}
async function callLLM(prompt) {
  const response = await
fetch("https://openrouter.ai/api/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${getAPIKey()}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      model: "meta-llama/llama-3-8b-instruct",
      messages: [
        {
          role: "user",
          content: prompt
        }
      ],
      temperature: 0.4
    })
  });
  const result = await response.json();
  if (!result.choices) {
    throw new Error("Invalid response from OpenRouter");
  }
  return result.choices[0].message.content;
}
/* ---------------- MOTIVATION ---------------- */
export async function generateMotivation(data) {
  const prompt = `
You are an academic performance coach.
Student metrics:
Streak: ${data.streak}
Productivity: ${data.productivity}
Stress Level: ${data.stress}/10
Generate a short motivational message.
Avoid generic quotes.
`;
  return await callLLM(prompt);
}
/* ---------------- TASK EXPLANATION ---------------- */
export async function generateExplanation(data) {
  const prompt = `
You are an AI study planner.
Explain why this study task is scheduled.
Subject: ${data.subject}
Weakness Score: ${data.weakness}
Exam in: ${data.examDays} days
Available Study Hours: ${data.availableHours}
Explain in 2-3 sentences.
`;
```

```
  return await callLLM(prompt);
}
/* ---------------- RESOURCE SUGGESTION ---------------- */
export async function generateResourceSuggestion(data) {
  const prompt = `
Suggest 1 or 2 learning resources for ${data.subject}.
Include:
- Course name
- Platform (YouTube, Coursera, edX, MIT OCW, Khan Academy)
- If possible include the course topic
Keep it concise.
`;
  return await callLLM(prompt);
}
/* ---------------- SCHEDULE INSIGHT ---------------- */
export async function generateScheduleInsight(data) {
  const prompt = `
You are an AI study coach.
Student Metrics:
Stress Level: ${data.stressLevel}/10
Productivity Score: ${data.productivityScore}
Schedule Confidence: ${data.confidenceScore}
Explain briefly how the study schedule was optimized.
Keep it motivational and informative.
`;
  return await callLLM(prompt);
}
```

File: `server.js`

Purpose:

Acts as the main entry point of the backend application. It initializes the Express server, loads environment variables, applies middleware, connects route files, and starts the server.

Overview:

This file prepares the backend so the frontend can communicate with the APIs. It is the first file executed when the backend starts.

Main Functions:

- Loads environment variables using dotenv
- Imports required modules like express, cors, and route files
- Creates the Express app
- Enables middleware such as cors() and express.json()
- Registers route modules like /api/genai
- Defines the server port
- Starts the backend server

Role in the Project:

It serves as the backbone of the backend system by handling server setup, configuration, API connection, and communication between frontend and backend.

Output:

When executed successfully, it starts the server and shows a confirmation message in the console.

Code:

```
import dotenv from "dotenv";
dotenv.config();
import express from "express";
import cors from "cors";
import genaiRoutes from "./routes/genai.js";
const app = express();
app.use(cors());
app.use(express.json());
app.use("/api/genai", genaiRoutes);
const PORT = 5000;
app.listen(PORT, () => {
  console.log(` Backend running on http://localhost:${PORT}`);
});
```

5.3.3 Frontend Implementation

File: `Analytics.tsx`

Purpose:

Implements the Analytics dashboard of the study planner and displays study performance, progress, subject-wise analysis, weekly activity, and achievements in a visual format.

Overview:

This component loads stored task data, calculates useful statistics, and presents them through cards, progress bars, charts, and badges. It helps users understand their study habits more clearly.

Main Functions:

- Loads task data using useState and useEffect
- Calculates key metrics such as completed tasks, total tasks, completion rate, total study time, and average task time
- Creates subject-wise task breakdown for bar chart visualization
- Calculates priority distribution for pie chart display
- Shows weekly progress using sample chart data
- Displays achievements based on completed task milestones

UI Elements:

- Header section

- Summary metric cards
- Subject-wise bar chart
- Priority pie chart
- Weekly progress line chart
- Achievement badges with progress bars

Special Note:

The weekly progress chart currently uses sample data for visualization and can be improved later to use real-time weekly tracking data.

Role in the Project:

It converts raw task data into meaningful analytics, helps students track their performance, and improves engagement through visual insights and gamification.

File: `Calendar.tsx`

Purpose:

Implements the Calendar module of the study planner and displays tasks in a weekly or monthly calendar format based on their due dates.

Overview:

This component loads saved study tasks and helps users view upcoming, ongoing, and completed tasks in a structured calendar layout. It supports weekly and monthly views, date navigation, and color-based task identification.

Main Functions:

- Loads stored task data using useEffect
- Stores task data, current date, and selected view using useState
- Filters tasks for a specific date using getTasksForDate()
- Renders a detailed weekly calendar using renderWeekView()
- Renders a compact monthly calendar using renderMonthView()
- Supports previous, next, and today navigation
- Allows switching between week and month view

UI Features:

- Header with title and subtitle
- Navigation controls for date movement
- Dynamic date label
- Weekly or monthly calendar display
- Color-coded tasks for priority and completion

- Legend explaining task colors

Task Colors:

- **Red** → High priority
- **Indigo** → Normal tasks
- **Green** → Completed tasks

Role in the Project:

This file helps organize tasks by real dates, improves schedule readability, and makes study planning more visual and user-friendly.

File: `Dashboard.tsx`

Purpose:

Implements the main dashboard of the AI-based study planner and provides an overview of study progress, upcoming tasks, AI recommendations, and quick navigation options.

Overview:

This component acts as the homepage of the application. It loads task data, calculates important metrics, generates AI-based recommendations, and displays everything in a structured and user-friendly interface.

Main Functions:

- Loads and initializes study task data
- Stores tasks and AI recommendations using React state
- Calculates completion rate, total study time, and today's tasks
- Identifies upcoming pending tasks based on due date
- Generates urgency labels such as Today, Tomorrow, and Overdue
- Applies visual styling based on task priority

UI Elements:

- Welcome header with study overview
- Summary statistic cards
- AI recommendations section
- Upcoming tasks list
- Quick action buttons for major features

Special Notes:

- Sample task data may be initialized for demonstration when no tasks are available
- The current streak value is shown as a static value and can be made dynamic in future versions

Role in the Project:

This file serves as the central control panel of the study planner by summarizing the user's study status, highlighting urgent tasks, showing AI suggestions, and improving navigation across the application.

File: `NotFound.tsx`

Purpose:

Implements the 404 error page shown when a user tries to access a route that does not exist.

Overview:

This component improves user experience by clearly informing the user that the requested page is unavailable and providing navigation options to return safely.

Main Functions:

- Displays a clear 404 error message
- Shows a short explanation that the page was not found
- Provides a **Go Home** button to return to the homepage
- Provides a **Go Back** button to return to the previous page

Role in the Project:

It handles invalid routes gracefully, prevents confusion, and improves navigation recovery.

File: `PomodoroTimer.tsx`

Purpose:

Implements the Pomodoro Timer feature of the study planner to help students manage focused study sessions and short breaks.

Overview:

This component provides an interactive timer linked to a selected task. It supports starting, pausing, resetting, switching between work and break sessions, saving completed Pomodoro sessions, and showing notifications.

Main Functions:

- Manages timer state using useState, useEffect, and useRef
- Supports 25-minute work sessions and 5-minute break sessions
- Starts, pauses, and resets the timer
- Automatically switches between work and break modes
- Saves completed study sessions using savePomodoroSession()

- Displays session completion and break reminders using toast notifications
- Formats time in MM:SS form for display

UI Elements:

- Header with task title and close button
- Large countdown timer
- Progress bar
- Start, Pause, and Reset controls
- Short Pomodoro technique explanation

Special Note:

The onComplete prop is defined for future completion handling, but it is not currently used inside the component.

Role in the Project:

This file adds a productivity-focused study tool, helps students maintain focus, records completed sessions, and improves the practical usefulness of the planner.

File: `Root.tsx`

Purpose:

Implements the root layout of the study planner and provides the common structure shared across all pages, including header, navigation, content area, and toast notifications.

Overview:

This component acts as the main application wrapper. It ensures a consistent layout throughout the system by rendering common navigation and dynamically displaying page content using React Router's Outlet.

Main Functions:

- Detects the current route using useLocation()
- Defines reusable navigation items for all major pages
- Displays desktop navigation in the header
- Displays mobile navigation at the bottom of the screen
- Highlights the active page
- Renders page-specific content through Outlet
- Enables global toast notifications using Toaster

UI Features:

- Application branding in the header
- Responsive navigation for desktop and mobile

- Shared main content container
- Global notification support

Role in the Project:

This file acts as the main application shell, keeps navigation consistent, supports responsive design, and reduces repeated layout code across pages.

Code:

File: `Settings.tsx`

Purpose:

Implements the Settings module of the study planner and allows users to manage profile details, notifications, study preferences, privacy options, and account-related actions.

Overview:

This component provides a centralized page for customizing application behavior and user preferences. It improves personalization and user control.

Main Functions:

- Stores settings using a structured useState object
- Manages notification, appearance, study, and privacy settings
- Uses a helper function to toggle boolean settings
- Provides editable profile fields
- Displays configurable notification options with switches
- Allows adjustment of study-related preferences such as Pomodoro duration
- Includes privacy controls and account action buttons

UI Features:

- Header section
- Profile settings card
- Notification preferences card
- Study preferences card
- Privacy controls
- Action buttons like Export Data, Delete Account, and Logout

Special Notes:

- Default profile values appear to be placeholder data
- pomodoroLength is connected to state, while some other study timing fields can be further extended

- Account action buttons are present in the interface, though they may not yet be fully connected to backend functionality

Role in the Project:

This file centralizes user preference management, supports personalization, and makes the study planner more flexible and user-friendly.

File: `StudyPlan.tsx`

Purpose:

Implements the Study Plan generation module of the AI-based study planner. It creates a personalized study schedule using task data, daily study hours, stress level, productivity, and AI-based enhancement.

Overview:

This component is one of the core modules of the application. It collects user planning inputs, generates a stress-aware schedule, sends it to the backend for AI enhancement, and forwards the final result to the AI Schedule page.

Main Functions:

- Loads saved tasks and progress data
- Stores daily study hours, stress level, schedule, and progress using React state
- Generates a stress-adjusted schedule from pending tasks
- Sends the generated schedule to the backend AI service for enhancement
- Calculates productivity score, confidence score, total pending time, and estimated completion days
- Saves updated progress and stores the final AI schedule in local storage
- Navigates to the AI Schedule page with enhanced schedule data
- Provides motivational support based on user streak and progress

UI Elements:

- Header section
- Summary statistics cards
- Progress card with streak and motivational quote
- Stress level slider and recommendation section
- AI schedule generator section
- Generate AI Schedule button
- Empty state when no pending tasks are available

Special Notes:

- The component includes an inline generated schedule display section, but in the current flow the final enhanced schedule is mainly forwarded to the dedicated AI Schedule page
- This makes the page act more as a planning and generation interface than the final display screen

Role in the Project:

This file acts as the intelligent planning engine interface of the application by combining scheduling logic, stress adaptation, progress tracking, and AI enhancement.

File: `Tasks.tsx`

Purpose:

Implements the Task Management module of the AI-based study planner. It allows users to create, view, filter, track, and delete study tasks, while linking task completion with Pomodoro sessions.

Overview:

This component serves as the execution layer of the system. It manages academic tasks, tracks real progress through Pomodoro-based work sessions, and integrates motivational feedback from the backend AI service.

Main Functions:

- Loads and normalizes saved task data
- Redistributes delayed tasks when needed
- Stores task list, active tab, selected task, and new task form data using React state
- Determines task status as Pending, In Progress, or Completed based on completed Pomodoro sessions
- Filters tasks by status using tab selection
- Adds new tasks through a dialog form
- Updates task progress after Pomodoro completion
- Calls the backend AI service to show motivational messages after study sessions
- Updates streak when a task is fully completed
- Deletes tasks and saves the updated list

UI Elements:

- Header section
- Add Task dialog
- Tabs for All, Pending, In Progress, and Completed tasks

- Task cards with status badges, due date, and progress bar
- Start Pomodoro and Delete buttons
- Pomodoro timer modal for selected tasks

Special Notes:

- Task completion is based on actual Pomodoro sessions rather than manual marking
- This makes progress tracking more realistic and productivity-focused

Role in the Project:

This file acts as the execution and progress tracking system of the study planner by managing tasks, recording focused work, updating completion status, and supporting motivation through AI feedback.

File: `study-data.ts`

Purpose:

Acts as the core data and utility module of the AI-based study planner. It defines the main data structures and provides logic for task storage, scheduling, productivity analysis, streak tracking, Pomodoro handling, delayed task redistribution, and recommendation support.

Overview:

This file serves as the central logic engine of the application. It manages local storage, study calculations, adaptive scheduling, progress tracking, and AI-style recommendation logic. Many major modules of the project depend on this file.

Main Functions:

- Defines core interfaces such as Task, AIScheduleItem, StudySession, StudyPlanData, UserProgress, and PomodoroSession
- Stores and retrieves tasks, sessions, and plans using local storage
- Initializes sample task data for first-time use
- Generates AI-style study recommendations from task patterns
- Detects the best study hour based on completed Pomodoro sessions
- Suggests a study schedule based on priority, due date, and daily hours
- Tracks user progress, streaks, and completed tasks
- Returns motivational quotes based on user condition
- Calculates schedule confidence score
- Provides stress-based study recommendations
- Analyzes subject weakness using task completion history
- Calculates productivity score

- Generates a stress-aware adaptive study schedule
- Redistributes overdue unfinished tasks to new dates
- Stores and loads Pomodoro session records

Special Importance:

The adjustScheduleForStress() function is the key intelligent scheduling algorithm in this file. It adapts session duration, break time, task ordering, and scheduling style based on stress level, productivity, subject weakness, priority, and urgency.

Role in the Project:

This file acts as the shared logic backbone of the study planner by connecting data storage, planning, progress evaluation, scheduling intelligence, and productivity tracking across multiple modules.

Code:

```
// Study data types and utilities

export interface Task {
  id: string;
  title: string;
  description: string;
  subject: string;
  priority: "low" | "medium" | "high";
  dueDate: string;
  completed: boolean;
  timeEstimate: number; // in minutes
  createdAt: string;
  isDelayed?: boolean;
  originalDueDate?: string;
}
export interface AIScheduleItem {
  task: Task;
  suggestedTime: string;
  sessionDuration: number;
  explanation?: string;
  recommendedCourse?: string;
}
export interface StudySession {
  id: string;
  taskId: string;
  date: string;
  duration: number; // in minutes
  completed: boolean;
}

export interface StudyPlanData {
  id: string;
  name: string;
  subjects: string[];
```

```
  startDate: string;
  endDate: string;
  dailyHours: number;
  createdAt: string;
}

export interface UserProgress {
  currentStreak: number;
  bestStreak: number;
  totalCompleted: number;
  lastCompletedDate: string | null;
  stressLevel: number;
}

export interface PomodoroSession {
  id: string;
  taskId: string;
  duration: number;
  completed: boolean;
  startTime: string;
}

// AI recommendations for study scheduling
export function generateAIRecommendations(tasks: Task[]): string[] {
  const recommendations: string[] = [];

  const highPriorityTasks = tasks.filter(t => !t.completed && t.priority
=== "high");
  const overdueTasks = tasks.filter(t => {
    if (t.completed) return false;
    return new Date(t.dueDate) < new Date();
  });

  if (highPriorityTasks.length > 0) {
    recommendations.push(` You have ${highPriorityTasks.length} high-
priority task${highPriorityTasks.length > 1 ? 's' : ''} pending`);
  }

  if (overdueTasks.length > 0) {
    recommendations.push(` ${overdueTasks.length} task${overdueTasks.length
> 1 ? 's are' : ' is'} overdue. Consider rescheduling.`);
  }

  const upcomingTasks = tasks.filter(t => {
    if (t.completed) return false;
    const daysUntilDue = Math.ceil((new Date(t.dueDate).getTime() - new
Date().getTime()) / (1000 * 60 * 60 * 24));
    return daysUntilDue >= 0 && daysUntilDue <= 3;
  });

  if (upcomingTasks.length > 0) {
    recommendations.push(` ${upcomingTasks.length}
task${upcomingTasks.length > 1 ? 's' : ''} due in the next 3 days`);
```

```
  }

  // Study pattern recommendations
  const totalMinutes = tasks.reduce((sum, t) => sum + t.timeEstimate, 0);
  const avgSessionTime = Math.floor(totalMinutes / Math.max(tasks.length,
1));

  if (avgSessionTime > 90) {
    recommendations.push(" Break down longer study sessions into 45-60
minute blocks with breaks");
  }

  const subjects = [...new Set(tasks.map(t => t.subject))];
  if (subjects.length >= 3) {
    recommendations.push("Alternate between subjects to maintain focus and
retention");
  }

  if (recommendations.length === 0) {
    recommendations.push(" Great! You're all caught up. Consider planning
ahead for upcoming topics.");
  }

  return recommendations;
}
export function detectBestStudyHour(): number {
  const sessions = loadPomodoroSessions();

  if (sessions.length === 0) return 9;

  const hourCounts: Record<number, number> = {};

  sessions.forEach(session => {

    if (!session.completed) return;

    const hour = new Date(session.startTime).getHours();

    hourCounts[hour] = (hourCounts[hour] || 0) + 1;
  });

  let bestHour = 9;
  let maxCount = 0;

  Object.keys(hourCounts).forEach(hour => {

    if (hourCounts[Number(hour)] > maxCount) {

      maxCount = hourCounts[Number(hour)];
      bestHour = Number(hour);
    }
  });
```

```
  return bestHour;
}
// Smart scheduling algorithm
export function suggestStudySchedule(tasks: Task[], dailyHours: number):
{task: Task, suggestedTime: string}[] {
  const schedule: {task: Task, suggestedTime: string}[] = [];
  const incompleteTasks = tasks.filter(t => !t.completed).sort((a, b) => {
    // Sort by priority and due date
    const priorityWeight = { high: 3, medium: 2, low: 1 };
    const aPriority = priorityWeight[a.priority];
    const bPriority = priorityWeight[b.priority];

    if (aPriority !== bPriority) return bPriority - aPriority;
    return new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime();
  });

  let currentDay = new Date();
  let dailyMinutesUsed = 0;
  const dailyMinutesAvailable = dailyHours * 60;

  incompleteTasks.forEach(task => {
    if (dailyMinutesUsed + task.timeEstimate > dailyMinutesAvailable) {
      // Move to next day
      currentDay = new Date(currentDay.getTime() + 24 * 60 * 60 * 1000);
      dailyMinutesUsed = 0;
    }

    const baseHour = detectBestStudyHour();
const hour = baseHour + Math.floor(dailyMinutesUsed / 60);
const minute = dailyMinutesUsed % 60;

const suggestedTime =
  `${currentDay.toLocaleDateString()} at
${hour}:${minute.toString().padStart(2, '0')}`;

    schedule.push({ task, suggestedTime });
    dailyMinutesUsed += task.timeEstimate;
  });

  return schedule;
}

// Local storage utilities
const STORAGE_KEYS = {
  TASKS: 'studymate_tasks',
  SESSIONS: 'studymate_sessions',
  PLANS: 'studymate_plans'
};

export function saveTasks(tasks: Task[]): void {
  localStorage.setItem(STORAGE_KEYS.TASKS, JSON.stringify(tasks));
}
```

```
export function loadTasks(): Task[] {
  const data = localStorage.getItem(STORAGE_KEYS.TASKS);
  return data ? JSON.parse(data) : [];
}

export function saveSessions(sessions: StudySession[]): void {
  localStorage.setItem(STORAGE_KEYS.SESSIONS, JSON.stringify(sessions));
}

export function loadSessions(): StudySession[] {
  const data = localStorage.getItem(STORAGE_KEYS.SESSIONS);
  return data ? JSON.parse(data) : [];
}

export function savePlans(plans: StudyPlanData[]): void {
  localStorage.setItem(STORAGE_KEYS.PLANS, JSON.stringify(plans));
}

export function loadPlans(): StudyPlanData[] {
  const data = localStorage.getItem(STORAGE_KEYS.PLANS);
  return data ? JSON.parse(data) : [];
}

// Sample data for initial setup
export function initializeSampleData(): void {
  const existingTasks = loadTasks();
  if (existingTasks.length === 0) {
    const sampleTasks: Task[] = [
      {
        id: '1',
        title: 'Review Calculus - Derivatives',
        description: 'Study chain rule and product rule applications',
        subject: 'Mathematics',
        priority: 'high',
        dueDate: new Date(Date.now() + 2 * 24 * 60 * 60 *
1000).toISOString(),
        completed: false,
        timeEstimate: 60,
        createdAt: new Date().toISOString()
      },
      {
        id: '2',
        title: 'Physics Lab Report',
        description: 'Complete lab report on projectile motion experiment',
        subject: 'Physics',
        priority: 'high',
        dueDate: new Date(Date.now() + 3 * 24 * 60 * 60 *
1000).toISOString(),
        completed: false,
        timeEstimate: 90,
        createdAt: new Date().toISOString()
      },
      {
```

```
        id: '3',
        title: 'Read Chapter 5 - Chemistry',
        description: 'Chemical bonding and molecular structure',
        subject: 'Chemistry',
        priority: 'medium',
        dueDate: new Date(Date.now() + 5 * 24 * 60 * 60 *
1000).toISOString(),
        completed: false,
        timeEstimate: 45,
        createdAt: new Date().toISOString()
      },
      {
        id: '4',
        title: 'Practice Coding Problems',
        description: 'Solve 10 problems on arrays and strings',
        subject: 'Computer Science',
        priority: 'medium',
        dueDate: new Date(Date.now() + 4 * 24 * 60 * 60 *
1000).toISOString(),
        completed: true,
        timeEstimate: 120,
        createdAt: new Date(Date.now() - 2 * 24 * 60 * 60 *
1000).toISOString()
      }
    ];
    saveTasks(sampleTasks);
  }
}

// User progress tracking
const PROGRESS_KEY = 'studymate_progress';

export function loadProgress(): UserProgress {
  const data = localStorage.getItem(PROGRESS_KEY);
  return data ? JSON.parse(data) : {
    currentStreak: 0,
    bestStreak: 0,
    totalCompleted: 0,
    lastCompletedDate: null,
    stressLevel: 5
  };
}

export function saveProgress(progress: UserProgress): void {
  localStorage.setItem(PROGRESS_KEY, JSON.stringify(progress));
}

export function updateStreakOnTaskComplete(): UserProgress {
  const progress = loadProgress();
  const today = new Date().toDateString();
  const lastDate = progress.lastCompletedDate ? new
Date(progress.lastCompletedDate).toDateString() : null;
```

```
  if (lastDate === today) {
    // Already completed a task today, just increment total
    progress.totalCompleted++;
  } else {
    // New day
    const yesterday = new Date(Date.now() - 24 * 60 * 60 *
1000).toDateString();
    if (lastDate === yesterday) {
      // Streak continues
      progress.currentStreak++;
    } else if (lastDate !== today) {
      // Streak broken, reset
      progress.currentStreak = 1;
    }

    progress.bestStreak = Math.max(progress.bestStreak,
progress.currentStreak);
    progress.totalCompleted++;
    progress.lastCompletedDate = new Date().toISOString();
  }

  saveProgress(progress);
  return progress;
}

// Motivational quotes based on context
export function getMotivationalQuote(context: 'low_streak' |
'low_productivity' | 'encouragement' | 'start'): string {
  const quotes = {
    low_streak: [
      " Don't worry about the streak! What matters is that you're here
today.",
      " Every expert was once a beginner. Keep going!",
      " Small steps every day lead to big results.",
      " Your comeback story starts with this next task."
    ],
    low_productivity: [
      " Progress isn't always linear. Be patient with yourself.",
      " Even a little progress is still progress.",
      " Take a break if you need one. Rest is productive too!",
      " You've got this! One task at a time."
    ],
    encouragement: [
      " Great work! Keep up the momentum!",
      " You're making excellent progress!",
      " Your dedication is paying off!",
      " Consistency is key, and you're nailing it!"
    ],
    start: [
      " Start your streak today! Every journey begins with a single step.",
      " Ready to make today count? Let's begin!",
      " A new day, a new opportunity to learn!",
      " Let's turn goals into achievements!"
```

```
    ]
  };

  const contextQuotes = quotes[context];
  return contextQuotes[Math.floor(Math.random() * contextQuotes.length)];
}
//////////////////////////////////////////////////////////////
// NEW: Schedule confidence score
//////////////////////////////////////////////////////////////
export function calculateScheduleConfidence(tasks: Task[]): number {

  if (tasks.length === 0) return 0;

  const completedTasks = tasks.filter(t => t.completed);

  const completionRate =
    completedTasks.length / tasks.length;

  const progress = loadProgress();

  const streakFactor =
    Math.min(progress.currentStreak / 7, 1);

  const stressStability =
    1 - (progress.stressLevel / 10);

  ////////////////////////////////////////////////////////
  // NEW: consistency factor
  ////////////////////////////////////////////////////////

  let consistencyFactor = 0;

  if (progress.lastCompletedDate) {

    const lastDate =
      new Date(progress.lastCompletedDate);

    const today =
      new Date();

    const daysGap =
      Math.abs(
        (today.getTime() - lastDate.getTime())
        / (1000 * 60 * 60 * 24)
      );

    consistencyFactor =
      daysGap <= 1 ? 1 :
      daysGap <= 3 ? 0.7 :
      daysGap <= 7 ? 0.4 : 0.2;
  }

  ////////////////////////////////////////////////////////
```

```
  const confidence =
    completionRate * 40 +
    streakFactor * 25 +
    stressStability * 20 +
    consistencyFactor * 15;

  return Math.round(confidence);
}
// Stress-based AI recommendations
export function getStressBasedRecommendation(stressLevel: number): string {
  if (stressLevel >= 8) {
    return "AI Recommendation: Shorter sessions with more breaks";
  } else if (stressLevel >= 6) {
    return "AI Recommendation: Moderate sessions with regular breaks";
  } else if (stressLevel >= 4) {
    return "AI Recommendation: Standard session length";
  } else {
    return "AI Recommendation: You're relaxed! Optimize for longer focus
sessions";
  }
}
// Calculate subject weakness score based on past performance
export function calculateSubjectWeakness(tasks: Task[]): Record<string,
number> {
  const subjectStats: Record<string, { total: number; completed: number }>
= {};

  tasks.forEach(task => {
    if (!subjectStats[task.subject]) {
      subjectStats[task.subject] = { total: 0, completed: 0 };
    }

    subjectStats[task.subject].total++;

    if (task.completed) {
      subjectStats[task.subject].completed++;
    }
  });

  const weaknessScores: Record<string, number> = {};

  Object.keys(subjectStats).forEach(subject => {
    const { total, completed } = subjectStats[subject];

    const completionRate = total === 0 ? 0 : completed / total;

    // weakness = inverse of completion rate
    weaknessScores[subject] = 1 - completionRate;
  });

  return weaknessScores;
}
```

```
//////////////////////////////////////////////////////////////
// NEW: Calculate overall productivity score
//////////////////////////////////////////////////////////////

export function calculateProductivityScore(tasks: Task[]): number {

  const total = tasks.length;

  if (total === 0) return 0.5;

  const completed = tasks.filter(t => t.completed).length;

  const completionRate = completed / total;

  const progress = loadProgress();

  const streakFactor = Math.min(progress.currentStreak / 7, 1);

  const stressFactor = progress.stressLevel / 10;

  const score =
    (completionRate * 0.5) +
    (streakFactor * 0.3) +
    ((1 - stressFactor) * 0.2);

  return score;
}
// Adjust study schedule based on stress level
export function adjustScheduleForStress(
  tasks: Task[],
  dailyHours: number,
  stressLevel: number
): AIScheduleItem[] {
  const schedule: AIScheduleItem[] = [];

  // Adjust session duration based on stress

  const productivityScore = calculateProductivityScore(tasks);
  // Stress-based session duration
  let maxSessionDuration: number;
  let breakDuration: number;
  // Base stress adjustment
  if (stressLevel >= 8) {
    maxSessionDuration = 25;
    breakDuration = 10;
  }
  else if (stressLevel >= 6) {
    maxSessionDuration = 45;
    breakDuration = 10;
  }
  else if (stressLevel >= 4) {
    maxSessionDuration = 60;
    breakDuration = 15;
```

```
  }
  else {
    maxSessionDuration = 90;
    breakDuration = 15;
  }

  // NEW: Productivity adjustment (SAFE — does not remove tasks)
  if (productivityScore < 0.4) {

    maxSessionDuration *= 0.8;
    breakDuration += 5;

  }
  else if (productivityScore > 0.8) {

    maxSessionDuration *= 1.1;

  }

  // Clamp limits
  maxSessionDuration = Math.max(25, Math.min(maxSessionDuration, 120));
  // Get weakness scores
const weaknessScores = calculateSubjectWeakness(tasks);

const incompleteTasks = tasks
  .filter(t => !t.completed)
  .sort((a, b) => {

    const priorityWeight = { high: 3, medium: 2, low: 1 };

    const aPriority = priorityWeight[a.priority];
    const bPriority = priorityWeight[b.priority];

    // urgency score (closer deadline = higher urgency)
    const aUrgency = 1 / (new Date(a.dueDate).getTime() - Date.now() + 1);
    const bUrgency = 1 / (new Date(b.dueDate).getTime() - Date.now() + 1);

    // weakness score
    const aWeakness = weaknessScores[a.subject] || 0;
    const bWeakness = weaknessScores[b.subject] || 0;

    // final adaptive score
    const aScore =
      (2 * aPriority) +
      (3 * aWeakness) +
      (1 * aUrgency);

    const bScore =
      (2 * bPriority) +
      (3 * bWeakness) +
      (1 * bUrgency);

    return bScore - aScore;
```

```
  });

  let currentDay = new Date();
  let dailyMinutesUsed = 0;
  const dailyMinutesAvailable = dailyHours * 60;

  incompleteTasks.forEach(task => {
    let remainingTime = task.timeEstimate;

    while (remainingTime > 0) {
      const sessionTime = Math.min(remainingTime, maxSessionDuration);

      if (dailyMinutesUsed + sessionTime + breakDuration >
dailyMinutesAvailable) {
        currentDay = new Date(currentDay.getTime() + 24 * 60 * 60 * 1000);
        dailyMinutesUsed = 0;
      }

      const hour = 9 + Math.floor(dailyMinutesUsed / 60);
      const minute = dailyMinutesUsed % 60;
      const suggestedTime = `${currentDay.toLocaleDateString()} at
${hour}:${minute.toString().padStart(2, '0')}`;

      schedule.push({ task, suggestedTime, sessionDuration: sessionTime });

      remainingTime -= sessionTime;
      dailyMinutesUsed += sessionTime + (remainingTime > 0 ? breakDuration
: 0);
    }
  });

  return schedule;
}

// Detect and redistribute delayed tasks
export function redistributeDelayedTasks(tasks: Task[]): Task[] {
  const now = new Date();
  const updatedTasks = tasks.map(task => {
    if (!task.completed && new Date(task.dueDate) < now && !task.isDelayed)
{
      // Task is overdue, mark as delayed and reschedule
      const newDueDate = new Date(now.getTime() + 3 * 24 * 60 * 60 * 1000);
// 3 days from now
      return {
        ...task,
        isDelayed: true,
        originalDueDate: task.dueDate,
        dueDate: newDueDate.toISOString()
      };
    }
    return task;
  });
  return updatedTasks;
```

```
}
// Pomodoro session management
const POMODORO_KEY = 'studymate_pomodoro';
export function savePomodoroSession(session: PomodoroSession): void {
  const sessions = loadPomodoroSessions();
  sessions.push(session);
  localStorage.setItem(POMODORO_KEY, JSON.stringify(sessions));
}
export function loadPomodoroSessions(): PomodoroSession[] {
  const data = localStorage.getItem(POMODORO_KEY);
  return data ? JSON.parse(data) : [];
}
```

File: `AISchedule.tsx`

Purpose:

Implements the AI Enhanced Schedule module of the study planner and displays the final AI-improved study schedule with explanations, recommendations, and overall schedule insight.

Overview:

This component acts as the presentation layer for the generated schedule. It receives schedule data from the Study Plan page or local storage, sends it to the backend AI service for enhancement, fetches an overall insight, and displays the final explainable schedule to the user.

Main Functions:

- Loads schedule data from navigation state or local storage
- Sends the base schedule to the backend /api/genai/enhance-schedule endpoint
- Receives AI-generated explanations and recommended resources
- Requests an overall schedule insight from /api/genai/schedule-insight
- Stores enhanced schedule data in component state and local storage
- Handles fallback to the original schedule if AI enhancement fails
- Redirects the user to the related task page when a schedule item is clicked

UI Elements:

- Page title
- Loading state message
- Empty state message
- AI insight panel
- Interactive schedule cards showing task title, suggested time, duration, recommendation, and explanation

Special Note:

This file is the final intelligent schedule visualization layer of the project and strengthens explainability by showing both the generated schedule and the reason behind each study slot.

Role in the Project:

It connects planning output with AI enhancement and task execution, making it the main explainable schedule display module of the system.

Code:

```
import { useEffect, useState } from "react";
import { useNavigate, useLocation } from "react-router";
import { Card, CardContent } from "@/app/components/ui/card";
import { Badge } from "@/app/components/ui/badge";
import { Clock, BookOpen } from "lucide-react";
export function AISchedule() {
const navigate = useNavigate();
const location = useLocation();

const storedSchedule = localStorage.getItem("aiSchedule");

const initialSchedule =
  location.state?.schedule ||
  (storedSchedule ? JSON.parse(storedSchedule) : []);

const [schedule, setSchedule] = useState(initialSchedule);
const [insight, setInsight] = useState("");
const [loading, setLoading] = useState(true);
useEffect(() => {

  async function enhanceSchedule() {

    const stateData = location.state;

    if (!stateData || !stateData.schedule) {
      console.log("No schedule received");
      setLoading(false);
      return;
    }

    try {

      /* ---------- ENHANCE SCHEDULE ---------- */

      const response = await fetch(
        "http://localhost:5000/api/genai/enhance-schedule",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
```

```
    body: JSON.stringify(stateData)
  }
);

const enhanced = await response.json();

console.log("AI schedule received:", enhanced);

if (Array.isArray(enhanced)) {

  setSchedule(enhanced);

  localStorage.setItem(
    "aiSchedule",
    JSON.stringify(enhanced)
  );

} else {

  console.log("Fallback used");

  setSchedule(stateData.schedule);

}

/* ---------- AI SCHEDULE INSIGHT ---------- */

try {

  const insightResponse = await fetch(
    "http://localhost:5000/api/genai/schedule-insight",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        stressLevel: stateData.stressLevel,
        productivityScore: stateData.productivityScore,
        confidenceScore: stateData.confidenceScore
      })
    }
  );

  const insightData = await insightResponse.json();

  if (insightData.insight) {
    setInsight(insightData.insight);
  }

} catch (err) {

  console.log("Insight generation failed");
```

```
      }

    } catch (error) {

      console.log("AI enhancement failed", error);

      setSchedule(stateData.schedule);

    } finally {

      setLoading(false);

    }

  }

  enhanceSchedule();

}, [location.state]);

  if (loading) {
    return (
      <div className="p-8 text-center text-slate-600">
        Loading AI schedule...
      </div>
    );
  }

  if (schedule.length === 0) {
    return (
      <div className="p-8 text-center text-slate-600">
        No schedule found. Generate one first.
      </div>
    );
  }

  return (
    <div className="space-y-6">

      <h1 className="text-3xl font-bold">
        AI Enhanced Schedule
      </h1>

      {/* AI Insight Panel */}
      {insight && (
        <Card className="border-none shadow-sm bg-indigo-50">
          <CardContent className="p-5">

            <p className="font-semibold text-indigo-900 mb-1">
               AI Schedule Insight
            </p>
```

```
            <p className="text-sm text-indigo-800">
              {insight}
            </p>

          </CardContent>
        </Card>
      )}

      {schedule.map((item, index) => (
        <Card
          key={item.task.id}
          className="cursor-pointer hover:shadow-md transition"
          onClick={() =>
            navigate(`/tasks?start=${item.task.id}`)
          }
        >
          <CardContent className="p-6 space-y-3">

            <div className="flex justify-between items-center">
              <h2 className="font-semibold text-lg">
                {index + 1}. {item.task.title}
              </h2>

              <Badge>
                {item.suggestedTime}
              </Badge>
            </div>

            <p className="text-sm text-slate-600 flex items-center gap-1">
              <Clock className="w-4 h-4" />
              {item.sessionDuration} minutes
            </p>

            <div className="bg-indigo-50 p-3 rounded-lg">
              <p className="font-medium text-indigo-700 flex items-center
gap-1">
                <BookOpen className="w-4 h-4" />
                Recommended Course
              </p>

              <p className="text-sm">
                {item.recommendedCourse}
              </p>
            </div>

            <div className="bg-slate-50 p-3 rounded-lg">
              <p className="font-medium">
                Why This Slot?
              </p>

              <p className="text-sm text-slate-600">
                {item.explanation}
              </p>
```

```
            </div>

          </CardContent>
        </Card>
      ))}

    </div>
  );
}
```

File: `routes.tsx`

Purpose:

Defines the routing structure of the AI-based study planner and connects all major pages using React Router.

Overview:

This file acts as the central route configuration of the application. It maps URL paths to components, organizes child routes under the root layout, and handles invalid routes through the Not Found page.

Main Functions:

- Creates browser-based routing using createBrowserRouter
- Sets Root as the shared layout component
- Connects all main pages such as Dashboard, Study Plan, Tasks, Calendar, Analytics, Settings, and AI Schedule
- Defines the Dashboard as the default home route
- Handles unknown URLs using the NotFound component

Main Routes:

- / → Dashboard
- /plan → Study Plan
- /tasks → Tasks
- /calendar → Calendar
- /analytics → Analytics
- /settings → Settings
- /ai-schedule → AI Schedule
- * → Not Found page

Role in the Project:

This file acts as the navigation control layer of the application by managing page flow, route hierarchy, and invalid route handling.

Code:

```
import { createBrowserRouter } from "react-router";
import { Root } from "@/app/components/Root";
import { Dashboard } from "@/app/components/Dashboard";
import { StudyPlan } from "@/app/components/StudyPlan";
import { Tasks } from "@/app/components/Tasks";
import { Calendar } from "@/app/components/Calendar";
import { Analytics } from "@/app/components/Analytics";
import { NotFound } from "@/app/components/NotFound";
import Settings from "@/app/components/Settings";
import { AISchedule } from "@/app/pages/AISchedule";

export const router = createBrowserRouter([
  {
    path: "/",
    Component: Root,
    children: [
  { index: true, Component: Dashboard },
  { path: "plan", Component: StudyPlan },
  { path: "tasks", Component: Tasks },
  { path: "calendar", Component: Calendar },
  { path: "analytics", Component: Analytics },
  { path: "settings", Component: Settings },
  { path: "ai-schedule", Component: AISchedule },
  { path: "*", Component: NotFound },
],
  },
]);
```

The figure represents the layered architectural flow of the system. It explains how navigation and layout components guide the user to functional modules, how study-data.ts performs scheduling and storage operations, and how backend and AI layers collaborate to enhance schedules and return the final explainable output to the user.

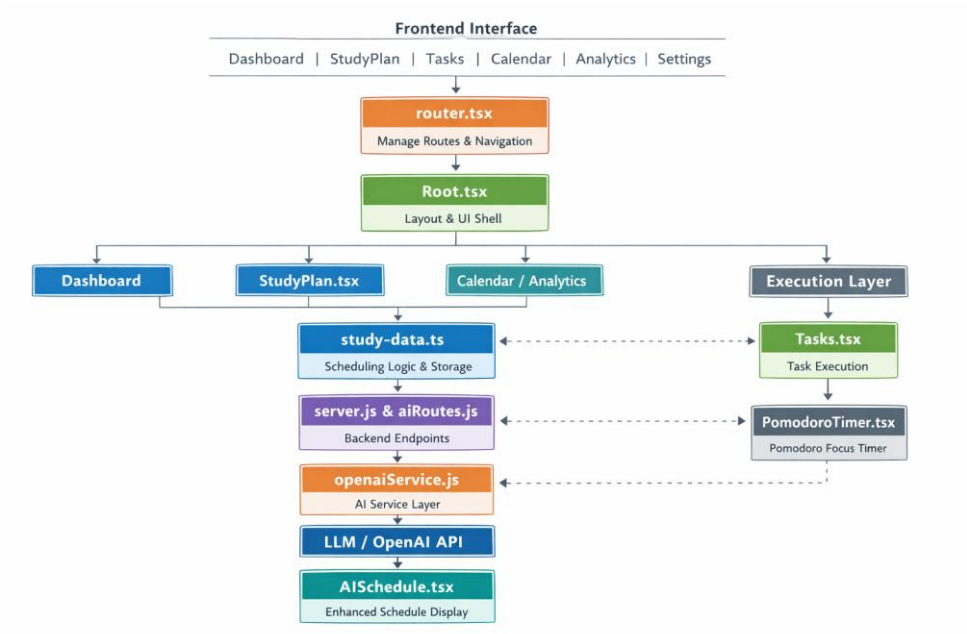

Fig 5.3.1 Layered Execution Flow of the AI-Based Study Planner

The figure illustrates the complete file-level interaction of the proposed AI-Based Study Planner. It highlights how user-facing modules communicate with the central logic and storage layer, which further interacts with backend GenAI routes and the external language model service to generate schedule explanations, recommendations, and motivational responses.

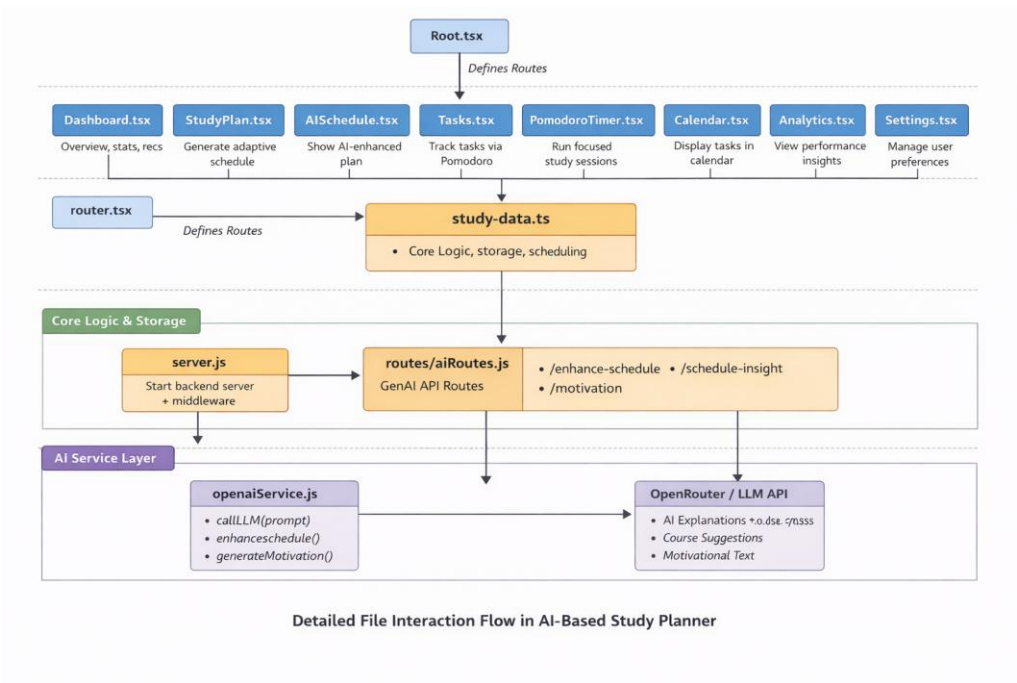

Fig. 5.3.2 File Interaction Flow of the AI-Based Study Planner

CHAPTER 6

TESTING

6.1 TEST CASES

Test cases were designed and executed to validate the major functionalities of the AI-Based Adaptive Study Planner. The purpose of these test cases was to ensure that the system performs accurately for various user inputs and operational scenarios.

Table 6.1: Test Cases of AI-Based Adaptive Study Planner

Test Case ID	Test Scenario	Input	Expected Output	Result
TC01	Add New Task	Valid task details	Task should be added successfully	Passed
TC02	Delete Task	Select existing task	Task should be removed	Passed
TC03	Generate Schedule	Input tasks and study hours	AI-generated schedule displayed	Passed
TC04	Stress Adjustment	High stress level input	Reduced workload generated	Passed
TC05	View Analytics	Completed tasks data	Analytics dashboard updates	Passed
TC06	Pomodoro Session	Start timer	Timer runs and records session	Passed
TC07	AI Recommendation	Request motivation/resource	AI response displayed	Passed

6.2 TESTING METHODS

6.2.1 Unit Testing

Unit Testing is the process of testing individual components or functions independently to ensure that each module performs correctly.

Examples of units tested include:

- Validation of task input fields
- Addition and deletion of study tasks
- Productivity score calculation
- Scheduling logic for task prioritization
- Routing and component-level behavior

Unit testing improved the reliability of individual modules and reduced the possibility of errors in later stages of development.

6.2.2 Integration Testing

Integration Testing is performed to verify whether different modules of the system work correctly when combined together.

The following integrations were tested:

- Connection between task management and schedule generation
- Integration of stored task data with analytics calculations
- Communication between frontend and backend APIs
- Navigation between dashboard, tasks, calendar, analytics, and settings

This testing ensured that all modules were properly connected and delivered a consistent user experience.

6.2.3 System Testing

System Testing is conducted to evaluate the complete and integrated software application as a whole.

System testing verified that:

- The application meets functional requirements
- The system behaves correctly under different user scenarios
- The complete workflow executes without failure
- The user interface responds properly across all modules

6.3 RESULTS AND ANALYSIS

The testing process showed that the AI-Based Adaptive Study Planner performed successfully across all major modules.

The results of testing indicated that:

- Task creation, update, and deletion were performed successfully
- Adaptive study schedules were generated accurately based on user input
- Stress-aware workload adjustments functioned properly
- Analytics correctly reflected user progress and productivity
- AI recommendations and schedule explanations were generated successfully
- Navigation between modules functioned smoothly

Overall, the system met the expected functional requirements and demonstrated satisfactory performance, reliability, and usability during testing.

CHAPTER 7

RESULTS&DISSCUSSIONS

The proposed Explainable AI-Based Adaptive Study Planner was implemented as a prototype web-based application designed to generate personalized and stress-aware study schedules. The system integrates task management, predictive intelligence, adaptive scheduling, and explainable AI components to support intelligent academic planning.

7.1 OUTPUT SCREENSHOTS

7.1.1 Dashboard Overview

The dashboard interface provides a summary of the user's study activity, including task completion rate, total study time, upcoming tasks, and AI recommendations. The dashboard serves as the central interface where users can monitor progress and access different features of the study planner.

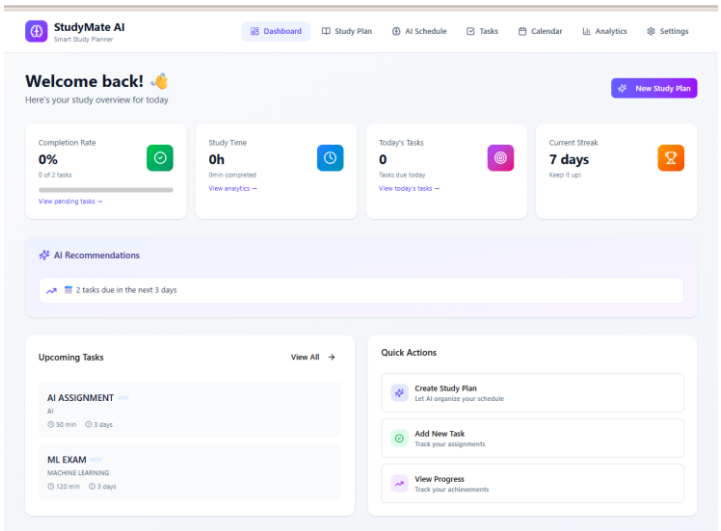

Figure 7.1.1: Dashboard Interface of the Study Planner

7.1.2 Task Management Interface

The task management module allows users to create and manage academic tasks by entering details such as subject name, priority level, deadline, and estimated study duration. The system organizes tasks into different states such as pending, in-progress, and completed, allowing users to track their study activities effectively.

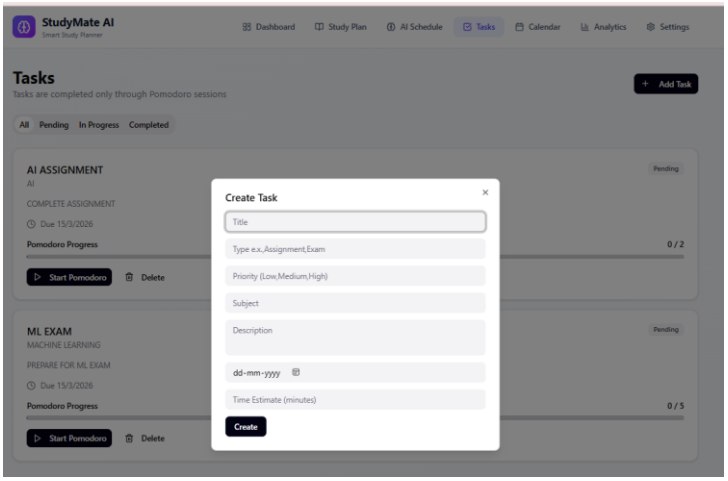

Figure 7.1.2: Task Creation and Task Management Interface

7.1.3 AI-Based Study Plan Generation

The study planning module generates personalized study schedules based on user inputs such as daily study hours, productivity score, and stress level. The adaptive scheduling engine analyzes these factors to create a balanced and achievable study plan.

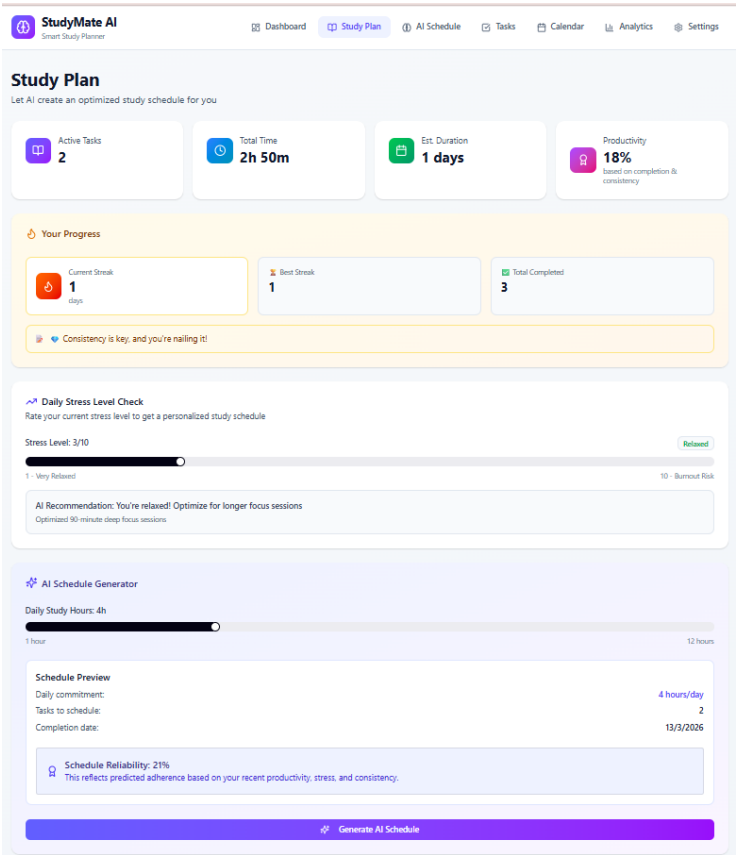

Figure 7.1.3: AI-Based Study Plan Generator

7.1.4 Calendar-Based Schedule Visualization

The generated study schedule is displayed through a calendar interface that allows users to view tasks distributed across different days. This feature helps students manage their academic workload and track scheduled study sessions efficiently.

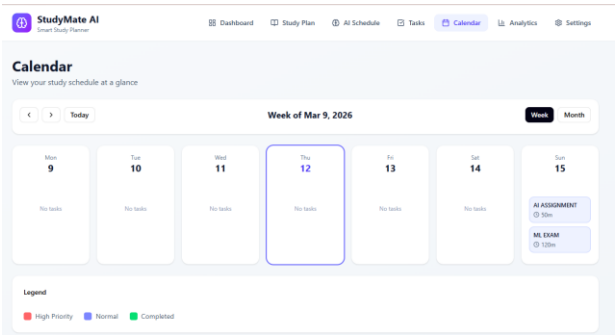

Figure 7.1.4: Calendar View of Generated Study Schedule

7.1.5 Study Analytics and Performance Insights

The analytics module provides visual insights into the user's learning behavior and performance patterns. The dashboard includes graphs showing task completion rate, study time distribution, subject-wise task analysis, and weekly productivity trends.

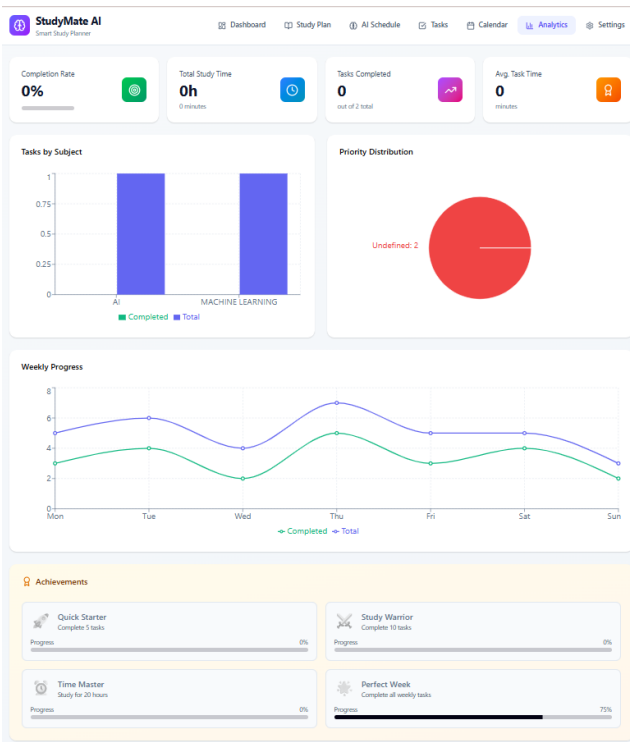

Figure 7.1.5: Study Analytics Dashboard

7.1.6 Explainable AI-Based Scheduling Output

The system also provides explainable insights into the generated study schedule. The Explainable AI component describes why specific tasks are scheduled at certain times and recommends relevant learning resources. This improves transparency and helps users understand the reasoning behind scheduling decisions.

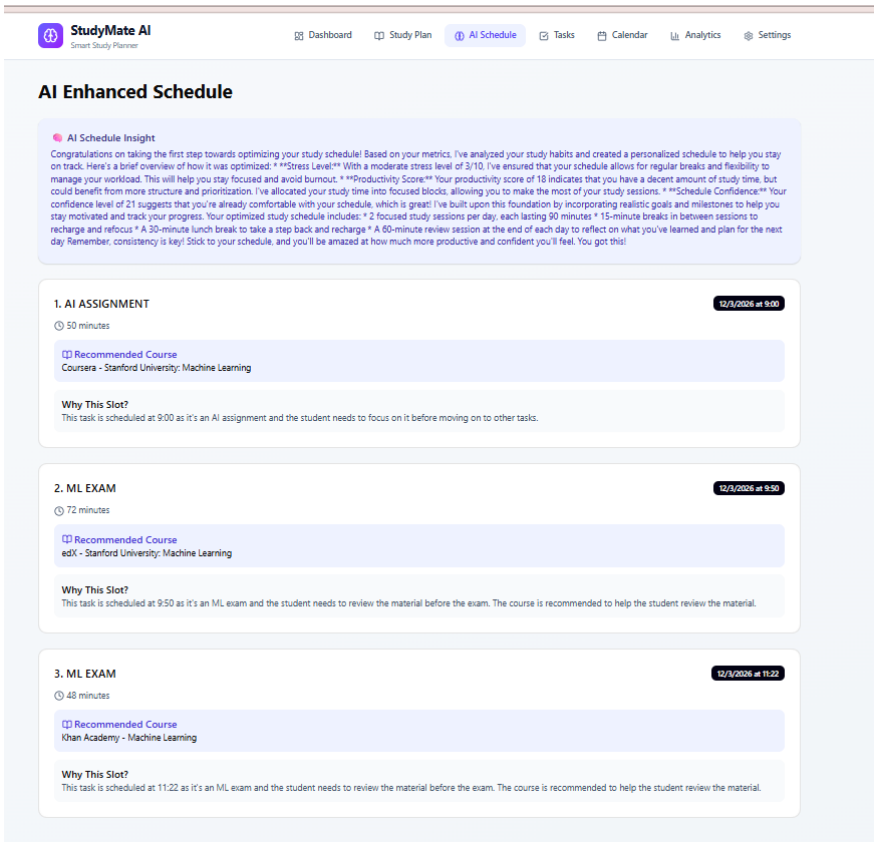

Figure 7.1.6: Explainable AI-Based Study Schedule Recommendations

7.2 PERFORMANCE ANALYSIS

The proposed AI-Based Adaptive Study Planner demonstrated effective performance during implementation and testing. The adaptive scheduling engine successfully generated balanced schedules based on task priority, stress level, and productivity metrics. The system effectively redistributed workload under varying stress conditions and maintained acceptable response time during schedule generation.

Performance observations include:

- Fast schedule generation and response time
- Accurate workload balancing based on stress levels
- Effective task prioritization using adaptive intelligence
- Smooth frontend-backend interaction with AI service integration

- Reliable analytics and dashboard rendering

The results indicate that the system efficiently performs intelligent study planning while maintaining usability and responsiveness.

7.3 COMPARISON WITH EXISTING SYSTEM

To evaluate the effectiveness of the proposed system, a comparison was made between traditional study planning approaches and the proposed AI-Based Adaptive Study Planner.

Table 7.1: Comparison of Traditional and Proposed Study Planning Systems

Metric	**Traditional Study Planning**	**Proposed AI-Based Planner**
Task Completion Consistency	Moderate due to static scheduling	Improved due to adaptive scheduling
Workload Balance	Often uneven and stressful	Balanced using stress-aware scheduling
Study Planning Efficiency	Manual planning requires more time	Automated AI scheduling improves efficiency
Personalization	Limited manual customization	High through behavioral analytics
Stress Management	Not supported	Integrated stress-aware workload adjustment
Productivity Tracking	Minimal or unavailable	Real-time productivity analytics available

The comparison shows that the proposed system significantly improves study consistency, workload distribution, efficiency, and personalization compared to traditional static planning methods.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 SUMMARY OF THE PROJECT

The proposed AI-Based Adaptive Study Planner was developed to provide students with an intelligent and personalized solution for academic schedule management. Unlike traditional static study planners, the system dynamically generates adaptive study schedules by considering task priorities, productivity patterns, stress levels, and user behavioral metrics.

The system integrates adaptive scheduling, wellbeing-aware planning, and explainable AI functionalities to enhance learning efficiency while maintaining student mental wellness. By combining predictive intelligence, workload balancing, and personalized study recommendations, the system transforms conventional study planning into an intelligent academic assistance framework.

Overall, the developed solution helps students improve study consistency, reduce academic pressure, maintain balanced workloads, and approach their academic goals with greater efficiency and confidence.

8.2 ACHIEVEMENTS

The major achievements of the proposed AI-Based Adaptive Study Planner include:

- Successful development of an intelligent adaptive study scheduling system
- Implementation of productivity-based and stress-aware workload adjustment
- Integration of explainable AI for schedule reasoning and study recommendations
- Development of real-time analytics dashboard for performance monitoring
- Implementation of Pomodoro-based wellbeing optimization mechanism
- Successful integration of frontend, backend, and AI service modules
- Creation of an interactive and user-friendly web-based interface

8.3 LIMITATIONS

Despite the effectiveness of the developed system, certain limitations exist in the current implementation:

- The system currently supports only single-user interaction and lacks multi-user authentication
- Data is stored using browser local storage instead of cloud/database storage
- The application is limited to web-based deployment and does not support native mobile platforms
- Stress level input currently depends on manual user entry rather than automated detection
- AI recommendations depend on internet/API availability for full functionality

8.4 FUTURE ENHANCEMENTS

Several enhancements can be incorporated in future versions of the proposed system to improve functionality and scalability:

- Integration of secure multi-user authentication and cloud database storage
- Development of a native mobile application with push notification support
- Implementation of cloud synchronization for cross-device accessibility
- Integration with wearable devices to collect biometric indicators such as heart rate, stress level, and sleep patterns
- Real-time adaptive scheduling using biometric and behavioral data
- Introduction of examination-focused intelligent revision planning features
- Addition of difficulty-aware scheduling for personalized academic preparation
- Enhancement of AI capabilities for deeper personalized educational guidance

REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed., Upper Saddle River, NJ, USA: Pearson Education, 2010.

[2] N. Annuš, "Software and Educational Artificial Intelligence: Students' Experiences and Innovative Solutions," *Information Technologies and Learning Tools*, vol. 101, no. 3, p. 200, 2024.

[3] H. R. Ahamed and D. K. Hanirex, "A Deep Learning-Enabled Approach for Real-Time Monitoring of Learner Activities in Adaptive E-Learning Environments," in *Proceedings of the 7th International Conference on Circuit Power and Computing Technologies (ICCPCT)*, IEEE, 2024, pp. 846–851.

[4] T. Zhang, *Exploring Children's Mathematics Learning and Self-Regulation in Robotics*, Doctoral dissertation, University of British Columbia, 2024.

[5] D. R. Kale and S. R. Todmal, "A Survey on Big Data Mining Applications and Different Challenges," *International Journal of Advanced Research in Computer Engineering and Technology*, vol. 3, pp. 3835–3838, 2014.

[6] J. A. Hendriks, *Exploring Study-Life Balance of Students and Their Perceptions of an Engagement App*, Bachelor's thesis, University of Twente, 2020.

[7] J. E. Nalavade, R. Sachdeo, D. R. Kale, A. Buchade, M. Subhedar, and S. K. Shinde, "Enhancing Road Safety: An Intelligent Drowsiness Detection System Based on Deep Neural Networks," in *Proceedings of the IEEE Pune Section International Conference (PuneCon)*, IEEE, 2024, pp. 1–6.

[8] S. Kim, "The Meaning of Personalized Learning Structures in AI-Based Educational Platforms: From the Perspective of Learned Curriculum," *Educational Technology International*, vol. 25, no. 2, pp. 297–329, 2024.

[9] G.Lingshetti, S.Pawar, M.Shinde, and H.Sayeed, "AI-Powered Smart Study Planner: Enhancing Personalized Learning Through Intelligent Scheduling."

[10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

[11] F. Cirillo, "The Pomodoro Technique," Creative Commons, 2006.

[12] P. Brusilovsky, "Adaptive Educational Systems on the World Wide Web: A Review of Available Technologies," International Journal of Artificial Intelligence in Education, vol. 13, no. 2, pp. 159–172, 2001.

[13] R. S. Baker and K. Yacef, "The State of Educational Data Mining in 2009: A Review and Future Visions," Journal of Educational Data Mining, vol. 1, no. 1, pp. 3–17, 2009.